

Prediction of Forestry Profit using Machine Learning

João Mey Pinto

Thesis to obtain the Master of Science Degree in

Computer Science and Engineering

Supervisors: Prof. Daniel de Matos Silvestre
Prof. Guilherme Henrique Caçador Ramos

May 2026

Declaration

I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

Acknowledgments

I would like to thank my parents for their friendship, encouragement and caring over all these years, for always being there for me through thick and thin and without whom this project would not be possible. I would also like to thank my grandparents, aunts, uncles and cousins for their understanding and support throughout all these years.

I would also like to acknowledge my dissertation supervisors Prof. Daniel de Matos Silvestre and Prof. Guilherme Henrique Caçador Ramos for their insight, support and sharing of knowledge that has made this Thesis possible.

Last but not least, to all my friends and colleagues that helped me grow as a person and were always there for me during the good and bad times in my life. Thank you.

To each and every one of you – Thank you.

This work was supported by national funds through FCT – Fundação para a Ciência e a Tecnologia, I.P., and, where applicable, co-funded by European Union funds under project UID/50008/2025 – Instituto de Telecomunicações (DOI: <https://doi.org/10.54499/UID/50008/2025>).

This research was also supported by CTS and LASI under grant UID/00066/2025, and by project gEICko-101223148-IST-ID (1801P.01521.1.01) at ISR.

Partial funding was provided by the transForm Agenda – Sub-project PIV.1.1 (project no. C644865735-00000007), under the Mobilizing Agendas for Business Innovation (Call No. 02/C05-i01/2021), through the Recovery and Resilience Plan (PRR) and the European NextGenerationEU funds.

Additional support from FCT was provided within the scope of the CORKNUT project (PCIF/MOS/0012/2019).

Abstract

Accurate workforce planning for pine cone harvesting is a critical challenge in the forestry sector, where manual yield estimation methods often lead to significant operational inefficiencies. Because *Pinus pinea* cones require approximately three years to mature, detecting early-stage strobili at the canopy apices provides a crucial predictive window for future harvests. This thesis proposes a systematic, two-stage methodology to automate strobili detection from high-resolution Unmanned Aerial Vehicle (UAV) imagery. First, a classical Open Source Computer Vision Library (OpenCV)-based pipeline was established, utilizing color-space thresholding, canopy Region of Interest (ROI) filtering, and multi-template matching, which served as a performance baseline with an F1-score of 0.45. Recognizing the limitations of heuristic methods in handling complex canopy environments and spectral occlusion, a secondary, deep learning-based pipeline was developed. This framework utilizes the You Only Look Once (YOLO)v8 architecture optimized via Slicing Aided Hyper Inference (SAHI) to perform sliced inference on high-resolution imagery, maintaining feature integrity for small-scale targets (20px). Validated on a manually annotated dataset, the deep learning approach achieved an F1-score of 0.99, demonstrating significant performance gains over the classical baseline. The contributions of this work include a scalable detection workflow capable of precise, three-year yield forecasting, providing a robust solution for precision forestry that balances computational efficiency with state-of-the-art detection accuracy.

Keywords

Strobili Detection; Yield Forecasting; Drone Imagery; You Only Look Once (YOLO)v8; Slicing Aided Hyper Inference (SAHI); Precision Forestry.

Resumo

O planeamento preciso da mão-de-obra para a colheita de pinhas é um desafio crítico no setor florestal, onde os métodos manuais de estimativa de produção conduzem frequentemente a ineficiências operacionais. Como as pinhas de *Pinus pinea* demoram aproximadamente três anos a amadurecer, a deteção de estróbilos em fase inicial nos ápices da copa fornece uma janela de previsão crucial para colheitas futuras. Esta tese propõe uma metodologia em duas fases para automatizar a deteção de estróbilos a partir de imagens de veículos aéreos não tripulados (VANT) de alta resolução. Primeiramente, foi implementado um pipeline clássico baseado em Open Source Computer Vision Library (OpenCV), utilizando segmentação por espaço de cor, filtragem da região de interesse da copa e *template matching*, estabelecendo uma linha de base com uma pontuação F1-score de 0.45. Reconhecendo as limitações dos métodos heurísticos na gestão de ambientes florestais complexos e oclusões, foi desenvolvido um segundo pipeline baseado em aprendizagem profunda. Esta abordagem utiliza a arquitetura You Only Look Once (YOLO)v8, otimizada através de Slicing Aided Hyper Inference (SAHI) para realizar inferência em fatias (*sliced inference*), preservando a integridade das características para alvos de pequena escala (20px). Validado num conjunto de dados anotado manualmente, o pipeline de aprendizagem profunda alcançou uma pontuação F1-score de 0.99, superando significativamente a linha de base clássica. As contribuições deste trabalho incluem um fluxo de trabalho de deteção robusto capaz de realizar previsões de produção a três anos, oferecendo uma solução escalável para a silvicultura de precisão.

Palavras Chave

Deteção de Estróbilos; Previsão de Produção; Imagens de Drones; You Only Look Once (YOLO)v8; Slicing Aided Hyper Inference (SAHI); Silvicultura de Precisão.

Contents

1	Introduction	1
1.1	Problem Statement	2
1.2	Motivation and Relevance	3
1.3	Objectives	4
1.4	Document Outline	5
2	Background	7
2.1	Forestry Context (Pine Cone Biology and Harvesting)	7
2.2	Computer Vision Foundations	9
2.2.1	Classical Image Processing	9
2.2.2	Deep Learning and Convolutional Neural Networks	9
2.3	Evaluation Metrics	10
3	Related Work	13
3.1	Evolution of UAV-Based Agricultural Monitoring	14
3.2	Yield Estimation using UAV Imagery	14
3.3	Classical Segmentation in Forestry	15
	A – Color-Space Transformations.	15
	B – Noise Reduction and Morphological Filtering.	15
	C – Limitations of Classical Approaches.	16
3.4	Deep Learning Approaches (YOLO)	16
3.4.1	Small Object Detection	16
3.4.2	Tiling and Sliced Inference	16
3.4.3	Data Augmentation for Aerial Imagery	18
3.5	Gap Analysis	18
4	System Architectures	21
4.1	Methodological Overview	21
4.2	Classical Heuristic Pipeline	22
4.2.1	Data Indexing	22

4.2.2	Pre-processing and Tiling	22
4.2.3	Segmentation and Morphological Cleanup	24
4.2.4	Candidate Proposal and Verification	24
4.2.5	Hyperparameter Tuning Space	25
4.3	Deep Learning Pipeline	26
4.3.1	YOLOv8 Architecture and Backbones	26
4.3.2	Loss Functions	26
4.3.3	Slicing Aided Hyper Inference (SAHI)	27
4.4	Implementation Environment	28
4.4.1	Dataset Splitting and Tiling	28
4.4.2	Data Augmentation Configuration	28
4.4.3	Software and Hardware Specifications	29
5	Evaluation and Results	31
5.1	Evaluation Protocol	31
5.2	Classical Pipeline Optimization	32
5.2.1	Region of Interest (ROI) Tuning	32
5.2.2	Segmentation Tuning	33
5.2.3	Candidate Proposal Optimization	33
5.3	Deep Learning Fine-Tuning Results	34
5.3.1	Training Convergence and Stability	34
5.3.2	Operational Parameter Optimization	36
5.4	Comparative Analysis	38
5.4.1	Detection Accuracy Comparison	40
5.4.2	Computational Inference Latency	40
6	Conclusions	41
6.1	Summary of Findings	41
6.2	Limitations and Future Work	42
	Bibliography	43
A	Source Code Excerpts	51
A.1	YOLOv8 Fine-Tuning Configuration	51
A.2	SAHI Inference Implementation	53
A.3	Evaluation and Metric Calculation	55

List of Figures

1.1	UAV-captured images of a <i>Pinus pinea</i> canopy highlighting strobili camouflage	3
1.2	High-level architecture of the proposed deep learning detection pipeline	4
2.1	Morphological characteristics of <i>Pinus pinea</i>	8
2.2	Comparison of classical and deep learning computer vision paradigms	10
3.1	Illustration of the SAHI methodology for high-resolution imagery	17
4.1	Sequential stages of the canopy ROI generation process	23
4.2	Visual comparison of RGB, HSV, and YCrCb color space representations	24
4.3	Visual demonstration of the background noise elimination process	25
4.4	The original YOLOv8 architecture	27
4.5	Visualization of the training batches and augmentation strategy	29
5.1	ROI optimization: Mean image coverage versus target retention	33
5.2	Segmentation optimization: Mean image coverage versus mean instance recall	34
5.3	Detection performance of the classical pipeline on a sample image	35
5.4	Candidate proposal optimization: Precision-Recall scatter plot	35
5.5	Training metrics for YOLOv8-Small over 150 epochs	36
5.6	Training metrics for YOLOv8-Nano over 150 epochs	37
5.7	F1-score performance curves across varying confidence thresholds for YOLOv8-Small and YOLOv8-Nano	38
5.8	Detection performance of the deep learning pipeline following SAHI inference	39

List of Tables

3.1	Comparison of literature gaps in forestry object detection.	19
4.1	Summary of implementation environment specifications.	30
5.1	Optimal inference parameters for deep learning architectures.	38
5.2	Inference latency benchmark (CPU-based execution).	40

Listings

A.1	YOLOv8 fine-tuning configuration with advanced data augmentation.	51
A.2	SAHI sliced inference integration for high-resolution macro-images.	53
A.3	Evaluation logic for computing Precision, Recall, and F1-score via IoU matching.	55

Acronyms

OpenCV	Open Source Computer Vision Library
ROI	Region of Interest
SAHI	Slicing Aided Hyper Inference
YOLO	You Only Look Once
UAV	Unmanned Aerial Vehicle
CNN	Convolutional Neural Network
HSV	Hue, Saturation, and Value
RGB	Red, Green, and Blue
YCrCb	Luminance, Chrominance-red, and Chrominance-blue
TP	True Positive
FP	False Positive
FN	False Negative
IoU	Intersection over Union
mAP	mean Average Precision
GPU	Graphics Processing Unit
CPU	Central Processing Unit
CIoU	Complete Intersection over Union
BCE	Binary Cross-Entropy
NMS	Non-Maximum Suppression
VRAM	Video Random Access Memory
VI_s	Vegetation Indices
NDVI	Normalized Difference Vegetation Index

PAN Path Aggregation Network

FPN Feature Pyramid Network

MS COCO Microsoft Common Objects in Context

Glossary

F1-score

A measure of a model's accuracy that strikes a balance between precision and recall. It is defined as the harmonic mean of precision and recall..... iii, v, ix, 2, 4, 5, 11, 32, 34, 35, 38, 40–42

GFLOP

Acronym for Giga Floating-Point Operation, representing one billion (10^9) operations. It is a standard metric used to quantify the computational complexity of a model, essentially measuring how much "work" the hardware must perform during a single forward pass 26

heliophilic

A botanical term describing plants that thrive in high-intensity sunlight 2, 8

1

Introduction

Contents

1.1 Problem Statement	2
1.2 Motivation and Relevance	3
1.3 Objectives	4
1.4 Document Outline	5

Forestry operations, specifically those involving pine cone harvesting, rely heavily on manual observation to estimate yields and allocate labor. This thesis addresses the challenge of automating the detection and quantification of developing strobili (immature pine cones) from high-resolution Unmanned Aerial Vehicle (UAV) imagery. The goal is to replace subjective manual counts with a deterministic, highly accurate computer vision pipeline capable of forecasting future harvests.

Accurate yield forecasting is critical for economic and operational efficiency in the forestry sector. Correia et al. demonstrated that the biological cycle of *Pinus pinea* and varying environmental stressors cause large temporal and spatial fluctuations in cone production [1]. Because *Pinus pinea* strobili require approximately three years to mature into harvestable cones, detecting them early at the top of the canopy provides forestry managers with a critical three-year predictive window. Precise automated estimation of these strobili allows operators to negotiate appropriate contracts in advance, preventing both

overstaffing, which inflates operational costs, and understaffing, which leads to delayed or incomplete harvests.

Detecting strobili in aerial images presents a challenging computer vision problem at multiple scales. Strobili are diminutive targets, often spanning merely 20 pixels across within a 5000-pixel image, which develop at the distal ends of branches and blend spectrally and morphologically with the surrounding green canopy. This visual ambiguity is further compounded by dense foliage occlusion, severe shadows, and dynamic illumination changes (Figure 1.1). Naive scaling or global thresholding approaches fail under these conditions because the target objects are too small and lack the distinct geometric features typically required for reliable object detection.

Previous agricultural vision systems often rely on multispectral data or target uniformly structured orchards, which do not translate well to the complex, heliophilic "umbrella" canopy structure of *Pinus pinea* [2, 3]. Tang et al. categorized extreme scale variations and category imbalance as primary hurdles in UAV imagery that traditional methods fail to overcome [4]. Furthermore, classical color-based segmentation approaches lack the semantic context necessary for robust detection in the wild [5]. Conversely, while deep learning models offer superior feature extraction, standard architectures forcibly downscale input images. Khorsand et al. noted that this downscaling shrinks 20-pixel objects to virtually zero pixels, rendering them undetectable by standard Convolutional Neural Networks (CNNs) [6].

This work proposes a systematic, two-phase methodology to overcome these scale and semantic challenges. Initially, a classical computer vision pipeline was developed utilizing Hue, Saturation, and Value (HSV) color-space thresholding, Region of Interest (ROI) canopy isolation, and multi-template matching, which established a baseline F1-score of 0.44. To transcend the limitations of these hand-crafted heuristics, a deep learning framework was subsequently implemented (see the architecture in Figure 1.2). This secondary pipeline leverages the You Only Look Once (YOLO)v8-Small architecture combined with Slicing Aided Hyper Inference (SAHI) [7]. By tiling the high-resolution imagery during both training and inference, the system preserves the spatial integrity of small targets. Evaluated on a manually annotated dataset, the deep learning pipeline achieved a final F1-score of 0.99. The primary limitation of the proposed system is its reliance on high-resolution data acquisition and the computational overhead associated with processing numerous overlapping tiles during inference.

1.1 Problem Statement

In commercial pine orchards, productivity and yield vary significantly due to physiological, environmental, and management factors. *Pinus pinea* possesses a complex reproductive cycle highly sensitive to climatic conditions, making the accurate forecasting of harvestable cones a difficult task [3]. Because mature pine cones are hidden deep within the tree's architecture, manual yield estimates frequently de-



Figure 1.1: UAV-captured images of a *Pinus pinea* canopy. The full-scale image (left) illustrates the broad canopy structure, while the zoomed-in crop (right) highlights the severe camouflage and extreme small scale of the target strobili developing at the branch apices.

viate from actual harvest totals. This discrepancy leads to inefficient workforce planning and complicates labor contract negotiations.

With the increasing availability and decreasing cost of drone-based imagery, forestry managers have access to vast amounts of aerial data that capture the upper canopy, where new strobili form. However, the manual inspection of these high-resolution images to count immature cones is prohibitively time-consuming and prone to human error. There is an immediate operational need for an automated solution that can process UAV imagery to effectively detect and estimate strobili abundance at the branch apices, thereby streamlining the harvest planning process years before the cones mature.

1.2 Motivation and Relevance

The importance of accurate yield estimation is reflected in the high economic value derived from pine cones, which are harvested primarily for pine nut production. Because the proposed pipeline detects strobili that require three years to mature, forestry managers can predict workforce needs up to three years in advance. This forecasting window allows operators to optimize labor contracts, stabilize supply chains, and minimize unproductive downtime.

Furthermore, Nisa et al. confirmed that despite recent advancements in deep learning, the "small and tiny object detection gap" persists in aerial remote sensing [8]. Addressing this gap specifically for forestry applications contributes to the broader field of precision agriculture. An automated, highly accurate detection pipeline allows operators to build precise historical yield maps based on early-stage

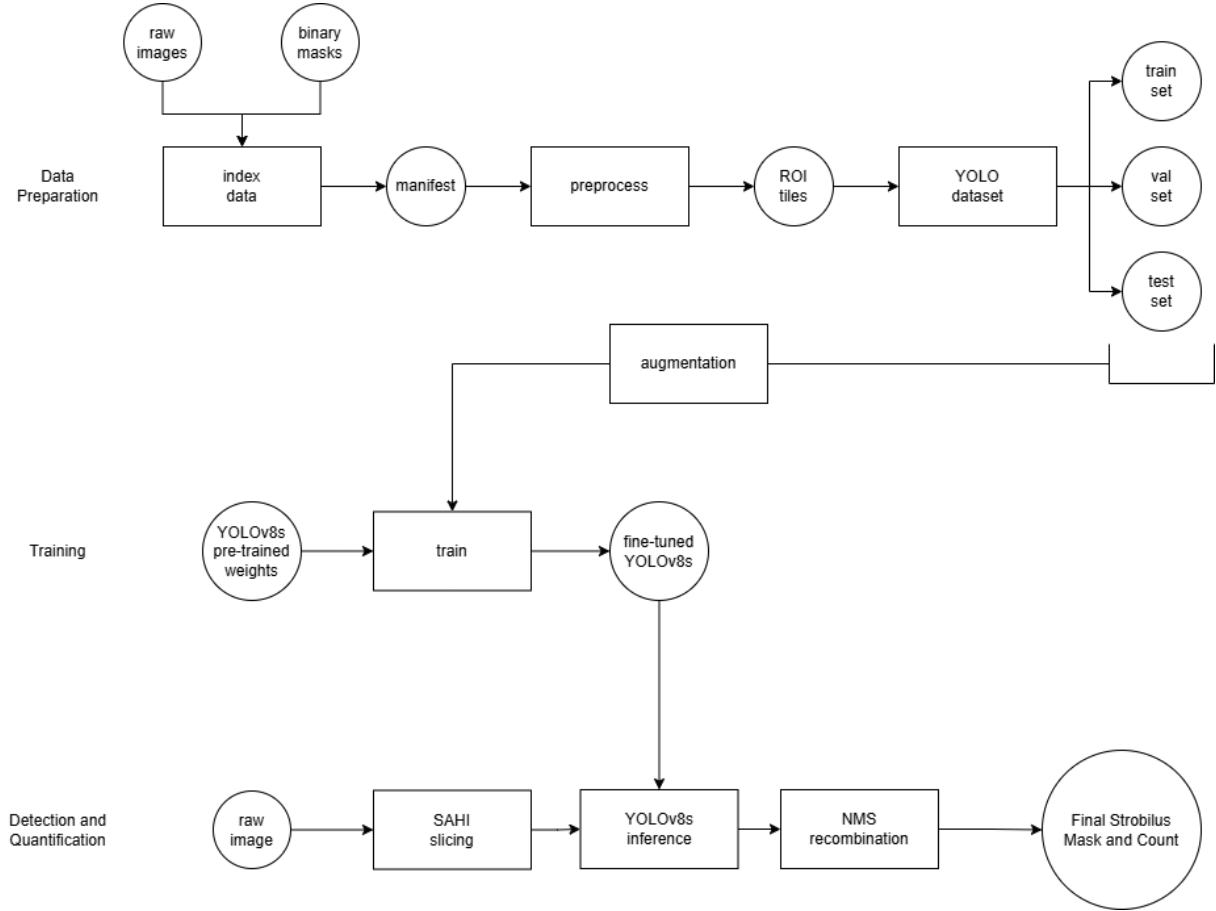


Figure 1.2: High-level architecture of the proposed deep learning detection pipeline, illustrating the transition from raw UAV imagery to sliced inference and final strobili aggregation.

strobili counts, directly supporting better inventory management and mitigating the financial risks associated with unexpected fluctuations in pine cone availability.

1.3 Objectives

The main objective of this thesis is to develop and rigorously evaluate an automated pipeline capable of detecting pine strobili in UAV imagery with an F1-score exceeding 0.90. To achieve this overarching goal and establish a transparent scientific comparison, the specific objectives are defined as follows:

1. **Baseline Establishment:** Develop a deterministic, Open Source Computer Vision Library (OpenCV)-based computer vision pipeline utilizing canopy ROI filtering, color-space thresholding, and morphological operations to quantify the performance ceiling of classical heuristics.
2. **Data Preprocessing and Augmentation:** Implement a rigorous image-level dataset split and a robust tiling methodology to maintain target resolution. Apply domain-specific photometric and

geometric data augmentations to simulate UAV flight variability and forest illumination changes [9, 10].

3. **Deep Learning Integration:** Fine-tune a modern CNN architecture (YOLOv8) specifically optimized for small-object feature retention, utilizing the prepared tile dataset.
4. **High-Resolution Inference:** Integrate the SAHI framework to perform sliced inference on full-scale 5K test images, ensuring that the dimensional integrity of the 20-pixel targets is preserved during the evaluation phase [7].
5. **Quantitative Evaluation:** Validate both pipelines against a manually annotated ground-truth dataset, reporting definitive metrics including Precision, Recall, and F1-score.

1.4 Document Outline

The remainder of this document is organized into five chapters, structured as follows:

Chapter 2 provides the theoretical background necessary to contextualize the research. The chapter details the biological characteristics of pine cone production, outlines the foundational concepts of classical and deep learning computer vision paradigms, and defines the formal metrics used to evaluate system performance.

Chapter 3 reviews the existing literature regarding automated agricultural monitoring. The chapter explores previous works in drone-based yield estimation, classical image segmentation, and modern deep learning approaches for small object detection. The chapter concludes with an analysis of the specific research gaps addressed by this thesis.

Chapter 4 details the methodologies and system architectures developed for strobili detection. The chapter outlines the specific data constraints, describes the internal mechanics of both the classical heuristic pipeline and the deep learning pipeline, and specifies the hardware and software environments utilized for implementation.

Chapter 5 presents the quantitative evaluation of the proposed pipelines. The chapter defines the strict dataset partitioning protocol, reports the optimization results of the classical grid searches, and details the fine-tuning outcomes of the deep learning models. Finally, the chapter provides a comparative analysis of detection accuracy and computational inference latency.

Chapter 6 summarizes the findings of the thesis. The chapter confirms the fulfillment of the primary research objectives, discusses the operational limitations of the developed systems, and proposes strategic directions for future work and real-world deployment.

2

Background

Contents

2.1 Forestry Context (Pine Cone Biology and Harvesting)	7
2.2 Computer Vision Foundations	9
2.3 Evaluation Metrics	10

This chapter contextualizes the challenge of strobili detection within the broader domains of precision forestry and computer vision. The chapter begins by detailing the biological and silvicultural characteristics of pine cone production to establish the economic and practical relevance of automated yield forecasting. Subsequently, the chapter explores the foundations of computer vision, contrasting classical heuristic pipelines with deep learning frameworks, to support the methodological choices made in this thesis. Finally, the formal evaluation metrics used to quantify system performance are defined.

2.1 Forestry Context (Pine Cone Biology and Harvesting)

The Mediterranean stone pine (*Pinus pinea*) is a species of significant economic importance, cultivated primarily for the production of highly valued pine nuts. However, managing *Pinus pinea* orchards



Figure 2.1: Morphological characteristics of *Pinus pinea*. The characteristic umbrella-shaped canopy (left) necessitates top-down UAV imaging, while the terminal development of strobili at the branch apices (right) introduces heavy visual camouflage against the foliage.

presents unique operational challenges due to the species' complex biology. As detailed by Correia et al., the tree exhibits a prolonged reproductive cycle, rendering cone production highly susceptible to fluctuations in soil conditions and climatic stressors [3]. These physiological and environmental factors cause substantial temporal and spatial variability in cone yield across different years and agricultural treatments [1].

Crucially, *Pinus pinea* exhibits terminal (distal) cone development, with strobili forming at the apices of the branches. These initial strobili require approximately three years to mature into harvestable pine cones. Because commercial pine cone harvesting remains a predominantly manual and labor-intensive process, forecasting this yield variability is essential for workforce planning. By detecting the strobili initially visible at the upper canopy, forestry managers can estimate the harvestable cone volume three years in advance, allowing them to secure appropriate labor contracts and allocate transportation resources efficiently.

To effectively automate this yield estimation using UAV imagery, a system must account for the specific morphology of the target trees. *Pinus pinea* is a heliophilic species characterized by wide horizontal branches that form a distinctive "umbrella" canopy shape [3]. While the umbrella canopy provides an advantageous top-down view for drone cameras, detecting mature cones is practically impossible, as they are hidden deep within the lower foliage branches. Therefore, aerial yield estimation relies exclusively on detecting the newly formed strobili at the upper canopy. Consequently, distinguishing these strobili, which share similar spectral properties with surrounding green needles and branch tips, requires image processing techniques capable of isolating the canopy region and filtering out background terrain.

2.2 Computer Vision Foundations

To solve the detection challenges posed by the *Pinus pinea* canopy, computer vision systems generally follow one of two paradigms: classical image processing or deep learning. Understanding the distinction between these paradigms justifies the methodological evolution presented in this thesis.

2.2.1 Classical Image Processing

Classical computer vision relies on deterministic, handcrafted algorithms to separate target objects from their background based on low-level features such as color, shape, and pixel intensity.

A primary step in classical pipelines is color-space conversion. While standard images are captured in the Red, Green, and Blue (RGB) color space, RGB channels are highly correlated and sensitive to illumination changes. To mitigate lighting variability, images are frequently transformed into spaces that separate luminance (intensity) from chrominance (color). Shaik et al. and Prasetyo et al. demonstrated that color spaces such as Luminance, Chrominance-red, and Chrominance-blue (YCrCb) and HSV provide superior robustness against non-uniform lighting when segmenting vegetation from background elements [11, 12]. Specifically, thresholding the chrominance channels enables the initial isolation of the green forest canopy from soil and roads.

Following thresholding, morphological operations are applied to refine the binary masks. Techniques such as erosion and dilation remove isolated noise pixels and consolidate fragmented object blobs, thereby improving overall segmentation accuracy [13, 14]. Once candidate regions are identified, template matching can be employed to score the regions against a predefined set of reference images.

Despite the computational efficiency of classical heuristics, these methods exhibit significant limitations in unstructured outdoor environments. Sadeghi et al. and Yu et al. showed that traditional color indices are highly sensitive to field-specific conditions, such as deep shadows and dynamic illumination [15, 16]. Furthermore, Wang et al. concluded that while color-based segmentation is useful for broad delineations, color thresholding fails to capture the higher-level semantic concepts required to distinguish objects with overlapping spectral signatures, such as developing strobili resting against complex green foliage [5].

2.2.2 Deep Learning and Convolutional Neural Networks

To overcome the limitations of handcrafted features, modern object detection relies on CNNs. Instead of using hardcoded color thresholds, CNNs learn hierarchical semantic features directly from annotated training data. Tian et al. demonstrated that deep learning frameworks successfully extract structural features that are not present in raw pixel intensity, thereby bridging the domain gap caused by variable forest lighting [17].

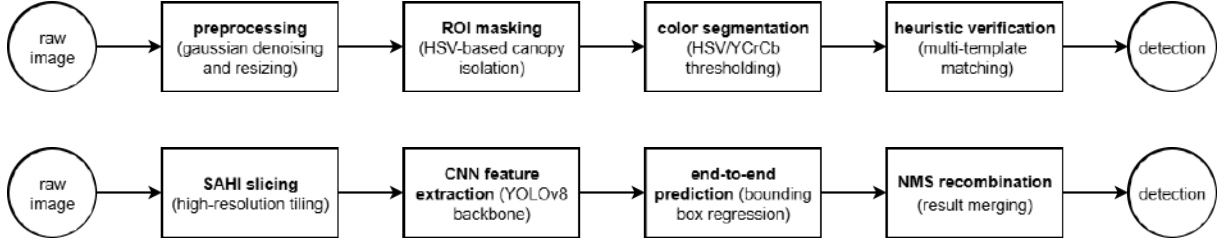


Figure 2.2: Comparison of computer vision paradigms. The classical pipeline (top) relies on sequential handcrafted heuristic filters, whereas the deep learning pipeline (bottom) utilizes end-to-end semantic feature learning.

However, the application of CNNs like the YOLO architecture to UAV imagery introduces a new specific challenge known as the "small object detection gap" [8]. Pine strobili often appear as pixel-sparse regions (e.g., 20×20 pixels) within massive 5472×3648 pixel aerial photographs. Khorsand et al. established that natively feeding such large images into standard CNN architectures forces the network to drastically downscale the input, shrinking small objects to near-zero dimensions and rendering the targets undetectable [6].

To preserve the spatial resolution necessary for small target detection, researchers employ image tiling and sliced inference techniques. By dividing large images into overlapping patches, models can process the data without destructive downsampling. Possolo et al. provided the mathematical foundation for exact tiling, demonstrating that calculated overlapping zones prevent the loss of features at tile seams [18]. During the inference phase, frameworks such as SAHI dynamically slice high-resolution images, process each slice independently, and subsequently merge the detections to form a complete output [7].

2.3 Evaluation Metrics

To quantitatively assess the performance of the detection pipelines, establishing formal evaluation metrics is necessary. The metrics used in this thesis are standard across object detection literature and rely on the classification of predictions into True Positives (TPs), False Positives (FPs), and False Negatives (FNs).

The foundational measurement for matching a predicted bounding box to a ground-truth annotation is the Intersection over Union (IoU). The IoU quantifies the spatial overlap between two bounding regions, defined mathematically as the area of intersection divided by the area of union [19]:

$$IoU = \frac{Area(Prediction \cap GroundTruth)}{Area(Prediction \cup GroundTruth)}. \quad (2.1)$$

A detection is registered as a True Positive if the IoU between the prediction and the ground truth exceeds a predefined threshold. Conversely, a False Positive occurs when the model predicts an object

where no ground truth exists, or the IoU falls below the threshold. A False Negative represents a ground-truth object that the model failed to detect.

Based on the counts of TPs, FPs, and FNs, the overall performance of the model is evaluated using Precision, Recall, and the F1-score [20, 21].

Precision measures the exactness of the model. Precision indicates the proportion of positive identifications that are actually correct:

$$Precision = \frac{TP}{TP + FP}. \quad (2.2)$$

Recall measures the completeness of the model. Recall indicates the proportion of actual ground-truth objects that were successfully identified:

$$Recall = \frac{TP}{TP + FN}. \quad (2.3)$$

The **F1-score** represents the harmonic mean of Precision and Recall. The F1-score provides a single, balanced metric that penalizes disparities between Precision and Recall, making the metric suitable for evaluating models operating on challenging datasets at specific confidence thresholds:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}. \quad (2.4)$$

While the F1-score provides a snapshot of performance at a fixed threshold, evaluating a deep learning model across all possible confidence thresholds requires a more comprehensive metric: the **mean Average Precision (mAP)**. The mAP is derived by calculating the area under the Precision-Recall curve. Because this thesis focuses on detecting a single object class (strobili), the Average Precision (AP) and mAP are equivalent.

Following the standard evaluation protocol established by the Microsoft Common Objects in Context (MS COCO) benchmark [22], the **mAP@50** metric is utilized. This metric denotes the mean average precision calculated at a strict IoU threshold of 0.5 (50% spatial overlap). The mAP@50 score serves as the primary benchmark for evaluating the convergence and overall localization accuracy of CNNs during the fine-tuning and validation phases.

These metrics provide a rigorous framework for quantifying detection accuracy and enable a direct performance comparison between the classical heuristic baseline and the deep learning architecture.

3

Related Work

Contents

3.1	Evolution of UAV-Based Agricultural Monitoring	14
3.2	Yield Estimation using UAV Imagery	14
3.3	Classical Segmentation in Forestry	15
3.4	Deep Learning Approaches (YOLO)	16
3.5	Gap Analysis	18

Research on automated crop and forest yield estimation has expanded significantly with advances in drone technology and computer vision. This chapter reviews the literature establishing the technical foundation for this thesis. Section 3.1 outlines the historical evolution of UAV-based agricultural monitoring. Section 3.2 discusses specific applications of UAV imagery for yield estimation in forestry. Section 3.3 examines classical image segmentation techniques and their inherent limitations. Section 3.4 explores modern deep learning approaches, focusing on small object detection, image tiling, and data augmentation. Finally, Section 3.5 synthesizes the existing literature to identify the primary research gaps addressed by the proposed architectures.

3.1 Evolution of UAV-Based Agricultural Monitoring

The integration of UAVs into agriculture and forestry has fundamentally transformed crop monitoring, enabling high-resolution spatial analysis that was previously impossible with ground surveys or satellite imagery [23]. Over the past decade, the analytical methodologies applied to UAV data have evolved from classical statistical approaches to advanced deep learning frameworks.

Historically, precision agriculture relied on classical remote sensing techniques, predominantly utilizing Vegetation Indices (VIs) such as the Normalized Difference Vegetation Index (NDVI) and static color thresholding [24]. While these classical methods excel at assessing generalized crop health and large-scale canopy vigor, they suffer from significant limitations when tasked with identifying specific, small-scale reproductive structures. Vidican et al. demonstrated that VIs are prone to "spectral confusion", where different plant components exhibit nearly identical reflectance signatures [25]. Furthermore, Wang et al. established that classical indices are highly sensitive to dynamic environmental factors, such as variable illumination and cloud cover, which frequently cause heuristic models to fail in unstructured outdoor environments [26].

To overcome the representation limitations of classical vegetation indices, the paradigm has shifted decisively toward deep learning [27]. Modern surveys of UAV-based disease and yield monitoring highlight CNNs, particularly single-stage detectors like YOLO and two-stage detectors like Faster R-CNN, as the current state-of-the-art [4]. These models learn hierarchical spatial features, allowing them to differentiate target objects from complex backgrounds regardless of lighting variations [28].

Despite this paradigm shift, the specific task of identifying individual, diminutive reproductive structures within dense forest canopies remains an underexplored niche. Feng et al. identified small object detection as one of the most persistent and challenging tasks in computer vision due to the inherently low resolution and noise representation of small targets in aerial imagery [29]. Similarly, Butler et al. noted that standard CNN architectures frequently lose crucial structural details when processing small objects against complex backgrounds, necessitating specialized architectural interventions [30]. This historical context underscores the necessity of transitioning from spectral-based classical segmentation to structurally aware deep learning pipelines, specifically tailored for the small-object detection constraints inherent in counting *Pinus pinea* strobili.

3.2 Yield Estimation using UAV Imagery

Early works on automated yield estimation predominantly focused on visually distinct crops, such as grapes and tomatoes. Researchers leveraged UAV imagery and classical computer vision techniques to detect these fruits against contrasting backgrounds [2]. However, translating these methods to forestry presents unique structural challenges.

In the context of *Pinus pinea*, Correia et al. detailed the biological dependencies and the extensive four-year reproductive cycle of the species, establishing that yield variability is highly dependent on physiological and climatic factors [3]. Correia et al. further addressed this variability by linking cone production outcomes to environmental treatments, highlighting the need for accurate aerial measurements to complement on-ground surveys [1]. Because *Pinus pinea* exhibits terminal cone development, with new strobili forming at the apices of the branches, UAV imagery is exceptionally well-suited for capturing the upper canopy.

To acquire useful data from these canopies, Humayun et al. demonstrated the value of building customized processing workflows for low-cost UAV platforms [31]. By combining the aerial perspective of UAVs with knowledge of terminal strobili development, forestry managers can forecast harvest yields up to three years in advance. However, accurately isolating these developing strobili from the surrounding foliage requires sophisticated image processing pipelines.

3.3 Classical Segmentation in Forestry

Before the widespread adoption of neural networks, forestry and agricultural vision systems relied heavily on classical image processing heuristics. These approaches generally segment images based on color-space transformations and morphological filtering.

A – Color-Space Transformations. Standard RGB images are highly sensitive to illumination changes. To mitigate lighting variability, researchers frequently convert images into color spaces that separate luminance from chrominance. Shaik et al. demonstrated that the YCrCb color space provides robust separation of color and intensity, making the space suitable for outdoor segmentation tasks [11]. Similarly, Prasetyo et al. found that the *Cr* channel in YCrCb consistently outperformed other color spaces when distinguishing vegetation from complex backgrounds [12]. Khan et al. also validated the necessity of evaluating multiple color spaces, such as HSV and YCrCb, to identify the most robust representation for specific agricultural environments [32].

B – Noise Reduction and Morphological Filtering. Outdoor UAV imagery frequently suffers from sensor noise and atmospheric artifacts. Martinez et al. identified noise reduction as a critical preprocessing step, proposing sequential non-linear filters to smooth images while preserving structural edges [33]. In addition to smoothing, researchers rely on morphological operations to refine binary masks. Abbas et al. utilized morphological opening and closing to enhance binary images and remove small, erroneous artifacts [13]. Furthermore, Miller et al. applied statistical filtering techniques to eliminate non-salient background points, a process analogous to removing roads and soil from aerial forest imagery to isolate

the canopy ROI [34].

C – Limitations of Classical Approaches. Despite computational efficiency, heuristic methods exhibit severe limitations. Wang et al. applied color-based segmentation to forest delineation and concluded that static color thresholds fail to capture the contextual understanding required to separate objects with overlapping spectral signatures [5]. Sadeghi et al. compared traditional color indices against machine learning classifiers, demonstrating that heuristic methods are highly sensitive to field-specific conditions, such as shading and illumination changes [15]. Yu et al. further corroborated that varying environmental illumination heavily degrades the accuracy of deterministic segmentation pipelines [16]. As a result, classical methods reach a performance ceiling when tasked with detecting camouflaged strobili under dynamic forest lighting.

3.4 Deep Learning Approaches (YOLO)

To address the feature extraction limitations of classical heuristics, the state-of-the-art in object detection has shifted to CNNs, particularly the YOLO architecture. However, deploying YOLO on drone imagery introduces challenges related to target scale, input resolution, and environmental variability.

3.4.1 Small Object Detection

Nisa et al. identified the "small and tiny object detection gap" as a persistent challenge in aerial remote sensing, noting that standard architectures struggle to extract features from objects encompassing only a few pixels [8]. Tang et al. categorized extreme scale variations as a primary hurdle that renders traditional deep learning methods ineffective without specific architectural modifications [4].

To address the small object gap, researchers have modified the standard YOLO feature extraction layers. Wang et al. introduced a dedicated P2 detection layer to the YOLOv8 architecture, preserving high-resolution feature maps specifically for small targets [35]. Similarly, Zeng et al. added a shallow detection head combined with spatial attention modules to prevent the structural detail of tiny objects from being lost during backbone downsampling [36]. Niu et al. optimized the network "neck" using an Asymptotic Feature Pyramid Network to manage the semantic gap between layers, ensuring that small targets remain distinct from background noise [37].

3.4.2 Tiling and Sliced Inference

Standard CNNs require fixed input dimensions (e.g., 640×640 pixels). Khorsand et al. demonstrated that directly downscaling $4K$ or $5K$ drone imagery to fit these dimensions shrinks 20-pixel objects to

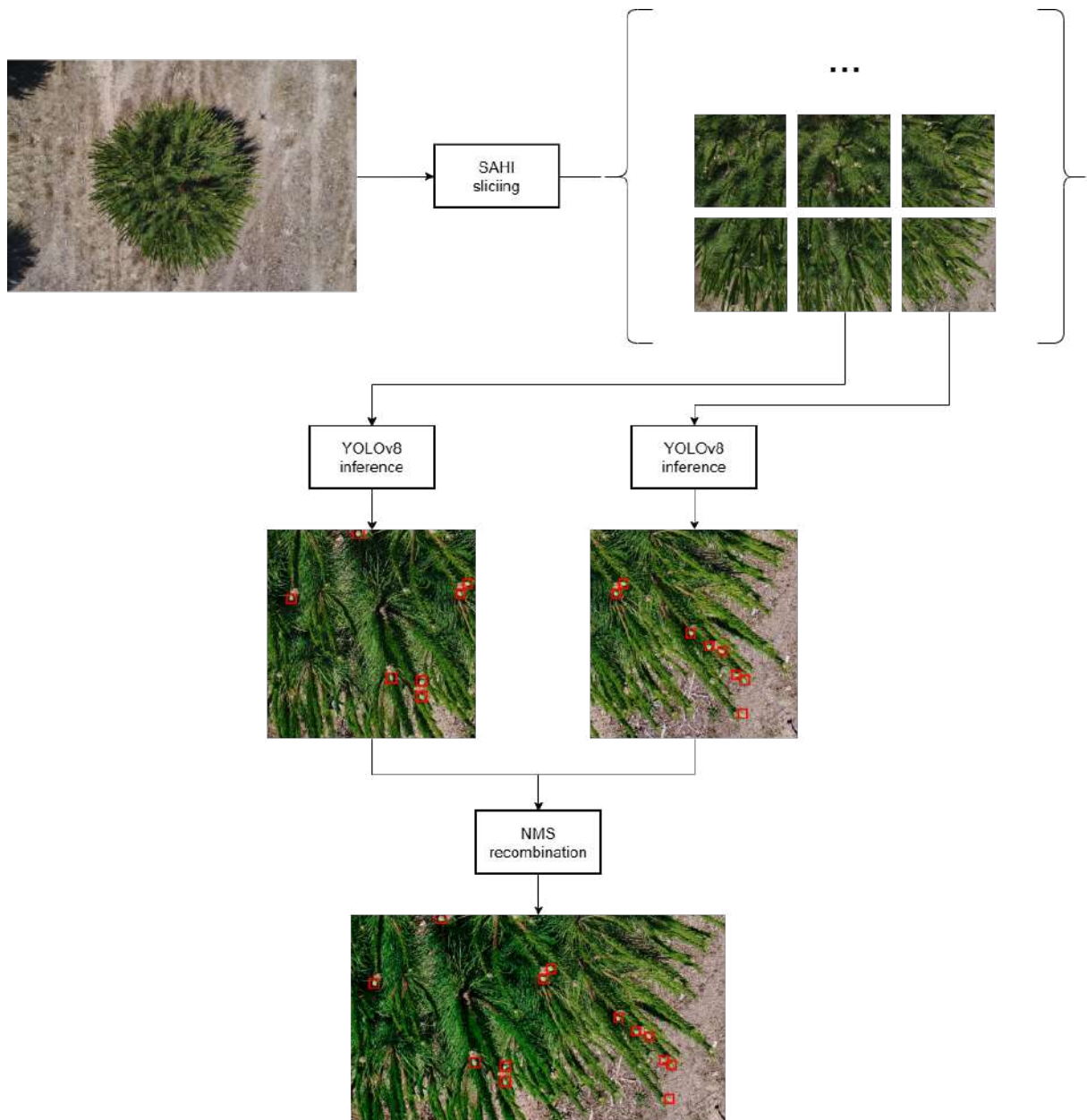


Figure 3.1: Illustration of the SAHI methodology, demonstrating the extraction of overlapping patches from a high-resolution source image, independent inference, and subsequent bounding box aggregation.

virtually zero pixels, making the objects completely undetectable [6].

To maintain native resolution, researchers employ image tiling. Possolo et al. established the theoretical foundation for error-free tiling, defining the specific overlap zones required to prevent artifacts at tile boundaries [18]. Building on this concept, Akyon et al. introduced the SAHI framework, a generic pipeline that performs automated slicing during inference and merges overlapping predictions using Non-Maximum Suppression [7]. Mazen et al. confirmed that the SAHI framework is model-agnostic and significantly enhances small object detection across various architectures [38].

Furthermore, exhaustive tiling over large orthomosaics is computationally expensive. Plastiras et al. demonstrated a selective tile processing mechanism that utilizes statistical information to process only "active" tiles containing relevant targets [39]. Zhu et al. introduced Label-Guided Optimal Tiling, proving that focusing computational effort solely on regions with high target probability yields superior inference throughput [40]. These selective tiling approaches provide theoretical justification for applying a Canopy ROI filter to discard empty background tiles before inference.

3.4.3 Data Augmentation for Aerial Imagery

Deep learning models require diverse datasets to generalize effectively. Kaur et al. emphasized the necessity of moving beyond simple geometric transformations (such as flipping and rotation) when addressing complex detection tasks [41].

To simulate environmental stressors, researchers rely on robust augmentation strategies. Yehia et al. utilized dynamic learning rates alongside advanced photometric augmentations to mitigate varying lighting and crop growth stages in aerial orchard imagery [42]. To address target occlusion and background clutter, Gishyan et al. investigated Grid Augmentation, which combines multiple images into a single training sample to artificially increase scene complexity [43]. Similarly, Luo et al. validated the Mosaic augmentation method, which stitches multiple image crops together to force the model to detect objects in highly cluttered environments [10]. These advanced augmentations are critical for training networks to recognize subtle visual cues under non-uniform forest illumination.

3.5 Gap Analysis

The literature review reveals distinct gaps in the current state of automated forestry yield estimation. Classical computer vision methods (Section 3.3) provide computational efficiency but lack the robustness required to detect camouflaged strobili under variable illumination. Conversely, while standard deep learning architectures (Section 3.4) offer superior semantic feature extraction, natively applying these architectures to high-resolution UAV imagery forces destructive downscaling, effectively erasing the small target objects.

Table 3.1: Comparison of literature gaps in forestry object detection.

Methodology	Identified Limitations	Proposed Mitigation in this Thesis
Classical OpenCV	Fails under dynamic illumination and spectral overlap [5, 16].	Transition to YOLOv8 for semantic feature learning.
Standard YOLO	Downscaling destroys small objects (<20px) [6].	Implementation of SAHI to maintain native image resolution.
Exhaustive Tiling	Computationally inefficient on massive drone imagery [40].	Canopy ROI filtering to discard non-target background tiles.
Basic Augmentation	Insufficient for handling complex forest shadows [41].	Photometric HSV shifts and Mosaic augmentation [42].

While recent research introduces frameworks like SAHI to mitigate scale issues, exhaustive sliced inference remains computationally intensive. Furthermore, much of the existing literature evaluates models on generalized datasets (e.g., VisDrone) rather than addressing the specific morphological traits of complex tree canopies like *Pinus pinea*.

Therefore, the main gap identified is the absence of an optimized, end-to-end pipeline that combines domain-specific preprocessing with high-resolution neural network inference. This thesis addresses this gap by integrating a classical Canopy ROI filter to reduce non-target computational load with a modern YOLOv8 architecture wrapped in the SAHI framework. This combination of selective preprocessing and sliced deep learning inference provides a solution tailored to the occluded, multi-scale challenge of strobili detection.

4

System Architectures

Contents

4.1 Methodological Overview	21
4.2 Classical Heuristic Pipeline	22
4.3 Deep Learning Pipeline	26
4.4 Implementation Environment	28

This chapter describes the two distinct system architectures developed to automate the detection of pine strobili. Section 4.1 outlines the core data constraints and image specifications common to the detection task. Section 4.2 describes the deterministic, heuristic-based classical computer vision pipeline. The subsequent sections detail the deep learning pipeline and the implementation environment used for both systems.

4.1 Methodological Overview

The detection of pine strobili from aerial photography imposes strict processing constraints due to the disparity between the input image resolution and the target object scale. The dataset consists of high-

resolution UAV imagery captured at 5472×3648 pixels. Within these images, individual strobili occupy regions as small as 20×20 pixels.

Processing images of this magnitude natively is computationally demanding for both classical and deep learning frameworks. Direct downscaling of the input images to standard processing resolutions (e.g., 640×640 pixels) degrades the spatial dimensions of the target objects to sub-pixel levels, making the strobili undetectable [6]. Consequently, any viable detection architecture must implement a localized processing strategy, such as image tiling, to evaluate the imagery at its native resolution while maintaining manageable memory footprints [44]. Furthermore, the images exhibit heavy foliage occlusion, background terrain visibility, and complex illumination patterns, requiring robust pre-processing mechanisms to isolate the relevant canopy elements before detection occurs.

Two parallel architectures were developed to address these constraints: a classical heuristic-based pipeline relying on explicit image processing rules, and a deep learning pipeline relying on convolutional feature extraction.

4.2 Classical Heuristic Pipeline

The classical computer vision pipeline is a deterministic, end-to-end system designed to detect strobili without the use of neural networks or learned weights. The architecture relies on sequential image processing techniques, combining color-space transformations, morphological filtering, and template matching. The pipeline is structured into four distinct modules: data indexing, pre-processing and tiling, segmentation, and candidate verification.

4.2.1 Data Indexing

The pipeline initiates with an automated data indexing module. This module scans the source directories containing the high-resolution raw images and their corresponding binary ground-truth masks. The script identifies pairs by matching shared filename stems. The paired paths are subsequently written into a centralized `manifest.csv` file. This manifest serves as the singular data reference for all downstream modules, ensuring that image arrays and ground-truth matrices remain strictly aligned during processing and evaluation.

4.2.2 Pre-processing and Tiling

The pre-processing module prepares the high-resolution imagery for segmentation by reducing noise and isolating the target canopy.

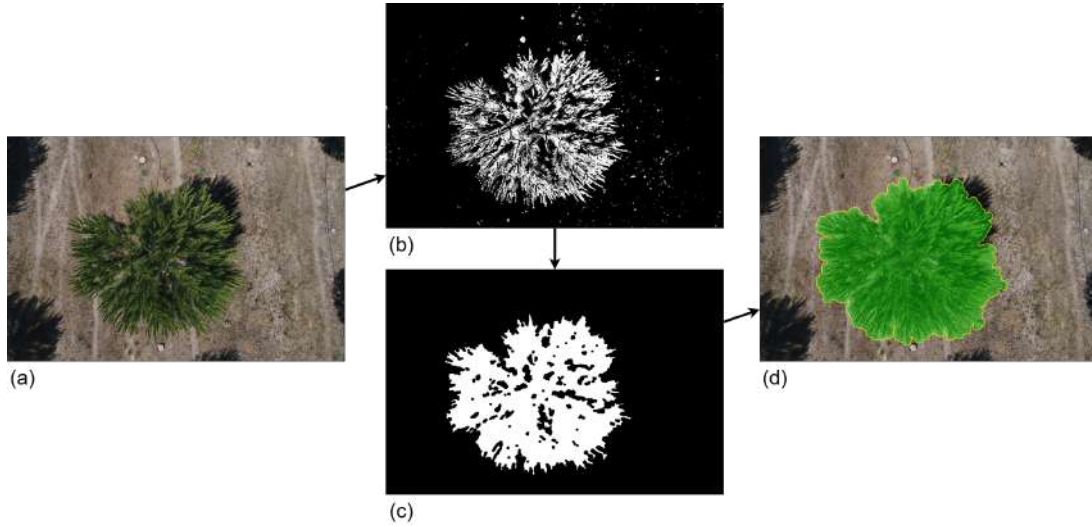


Figure 4.1: Sequential stages of the canopy Region of Interest (ROI) generation process: (a) original image resized to 2048×1365 pixels; (b) binary output of the green-hue thresholding in the HSV color space; (c) isolation of the largest connected component to identify the dominant tree; and (d) final dilated mask overlaid on the source imagery used to eliminate non-target background structures.

First, the source images are resized from 5472×3648 pixels to a standardized dimension of 2048×1365 pixels. A Gaussian filter is then applied to mitigate sensor artifacts and high-frequency noise, which is a standard procedure for improving the extraction of structural features in forest imagery [45].

Next, a Region of Interest (ROI) mask is generated to restrict subsequent processing strictly to the forest canopy, discarding background elements such as soil, roads, and sky. The ROI generation process involves:

1. Thresholding the image within the HSV color space to isolate broad green hues [14].
2. Applying a morphological closure operation to fill structural gaps within the thresholded foliage [13].
3. Extracting the largest connected component to isolate the dominant tree canopy.
4. Applying a morphological dilation to expand the boundary, ensuring that the peripheral branch apices where strobili develop are fully encompassed by the mask.

To facilitate fine-grained segmentation without exceeding memory constraints, the resized images and the corresponding ROI masks are decomposed into uniform spatial patches. The tiling operation extracts 608×608 pixel patches with a 32-pixel overlap, an approach proven effective for managing hardware limitations in UAV object detection [46]. The tiles are exported and stored in both HSV and YCrCb color spaces (see Figure 4.2), alongside the corresponding binary ROI patches.

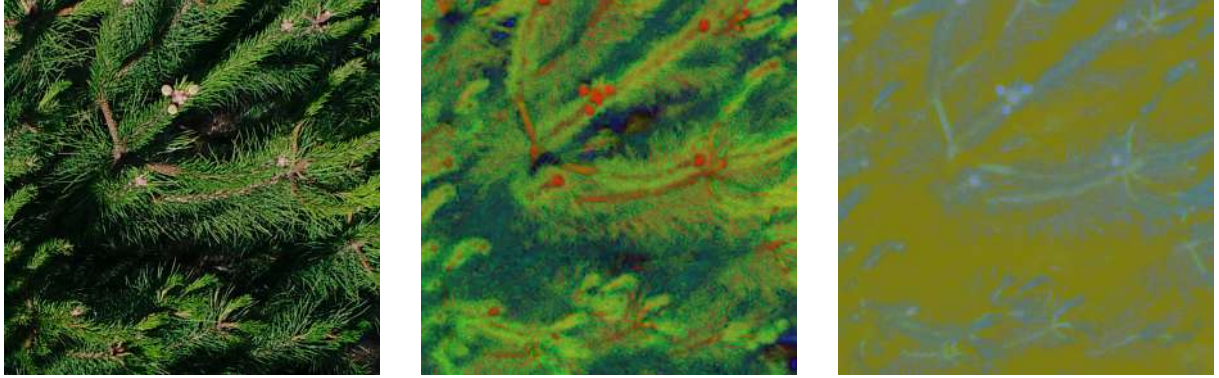


Figure 4.2: Visual comparison of color space representations for a 608×608 pixel tile. From left to right: original RGB, HSV, and YCrCb.

4.2.3 Segmentation and Morphological Cleanup

The segmentation module operates at the tile level to separate candidate strobili pixels from the surrounding canopy. Each tile is processed using hardcoded color-space threshold boundaries within the HSV space, leveraging its robustness against non-uniform outdoor lighting [11, 47].

To eliminate false positive pixels originating from the background terrain, the thresholded binary tile is logically intersected (via a bitwise AND operation) with its corresponding ROI mask patch (as illustrated in Figure 4.3). Once all tiles belonging to a single source image are processed, the segmented patches are composited back into a full-image segmentation mask.

Because color-space thresholding often produces fragmented pixel clusters, a global morphological cleanup phase is executed on the composited mask. A morphological closing operation (dilation followed immediately by erosion) consolidates fragmented blobs and eliminates minor, isolated pixel noise [48]. This process results in cohesive binary shapes representing potential strobili.

4.2.4 Candidate Proposal and Verification

The final module transforms the continuous binary segmentation mask into discrete bounding box predictions through a proposal and verification mechanism.

Proposal Generation: A sliding-window algorithm scans the full-image segmentation mask. The algorithm calculates the density of white pixels (positive segmentations) within the window area. If the calculated density exceeds a predefined threshold, the window coordinates are extracted as a candidate proposal box.

Multi-Template Verification: To filter out dense foliage erroneously flagged by the segmentation step, each candidate proposal is verified against a predefined library of 24 manually cropped strobili templates. The verification relies on the `cv2.matchTemplate` function utilizing the `TM_CCOEFF_NORMED` cross-correlation metric. Because proposals vary in size, the verification function dynamically scales the

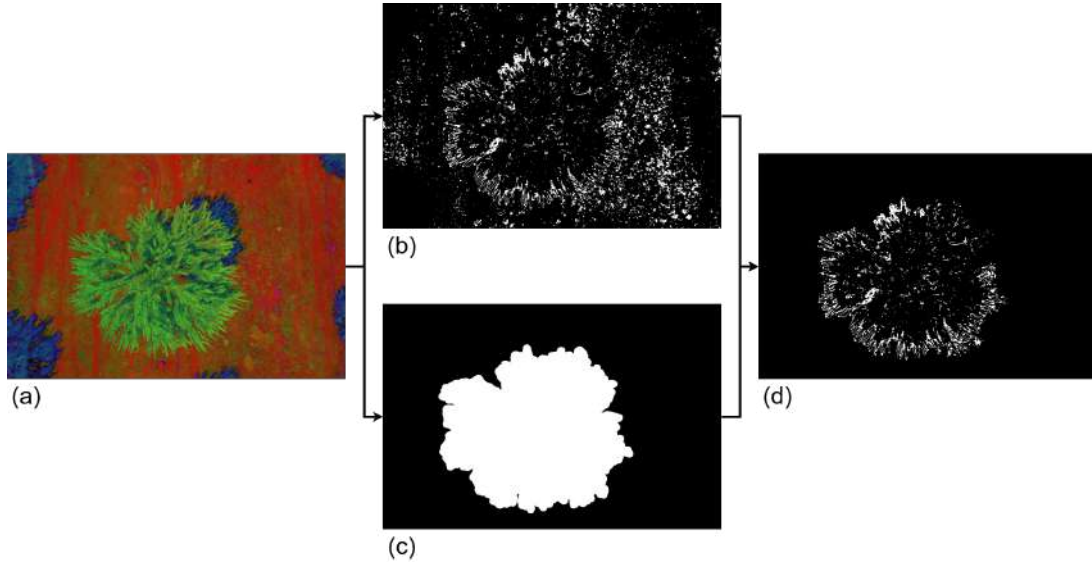


Figure 4.3: Visual demonstration of the background noise elimination process: (a) source image transformed into the target color space (HSV); (b) raw thresholding output containing significant false-positive noise from background terrain; (c) binary canopy ROI mask used as a spatial filter; and (d) final segmented output after a bitwise AND operation, isolating the target foliage for subsequent candidate proposal.

templates to match the dimensional bounds of the proposal window, ensuring robust cross-correlation scoring. Proposals yielding a match score below the configured threshold are discarded.

Non-Maximum Suppression: The sliding-window approach inherently generates multiple highly overlapping proposals for a single target. A Non-Maximum Suppression (NMS) algorithm is applied to the verified bounding boxes. The NMS algorithm evaluates the IoU of overlapping boxes and suppresses redundant predictions [19]. This ensures that only the bounding box with the highest template-matching score is retained for each distinct spatial region.

4.2.5 Hyperparameter Tuning Space

The classical pipeline relies on numerous explicit parameters. The architecture was configured to support exhaustive grid-searching via a centralized `default.yaml` configuration file. The tunable parameter space includes:

- **ROI Generation:** HSV lower and upper boundaries for canopy isolation, morphological kernel dimensions, and dilation iterations.
- **Segmentation:** Explicit HSV and YCrCb channel bounds, along with structural element shapes (e.g., elliptical versus rectangular) for morphological closing.
- **Verification:** Sliding-window pixel density thresholds, cross-correlation score minimums for template matching, and the IoU overlap thresholds governing the NMS algorithm.

4.3 Deep Learning Pipeline

The second architecture replaces the heuristic image processing rules with a convolutional neural network capable of extracting semantic features directly from the data. This pipeline is built upon the YOLOv8 object detection framework [49], augmented by a SAHI mechanism to address the extreme scale disparity of the strobili [7].

4.3.1 YOLOv8 Architecture and Backbones

YOLOv8 is a single-stage object detector. Unlike earlier iterations that relied on anchor boxes, YOLOv8 employs an anchor-free detection mechanism, directly predicting the center of an object and its bounding box dimensions. This reduces the number of heuristic parameters and improves generalization across varying object sizes.

The core architecture consists of a CNN backbone for feature extraction, a neck for multi-scale feature fusion, and a decoupled head. The backbone introduces the Cross-Stage Partial bottleneck with two convolutions (C2f module). This module optimizes gradient flow and improves feature representation for small targets without significantly inflating the computational cost [35]. The decoupled head separates the classification and bounding box regression tasks into parallel convolutional branches, reducing feature misalignment (see Figure 4.4).

Two specific backbone variants were evaluated in this architecture to explore the trade-off between computational efficiency and detection accuracy [51]:

- **YOLOv8-Nano** (`yolo8n`): The most lightweight configuration, utilizing aggressive depth (0.33) and width (0.25) multipliers. It contains approximately 3.2 million parameters and requires 8.7 GFLOPs at a 640×640 resolution. It is designed for maximum computational efficiency and deployment on resource-constrained edge devices.
- **YOLOv8-Small** (`yolo8s`): A slightly larger configuration containing approximately 11.1 million parameters and requiring 28.4 GFLOPs. This variant provides enhanced feature extraction capacity, offering a stronger representation for small targets while remaining compatible with standard consumer-grade Graphics Processing Units (GPUs).

4.3.2 Loss Functions

During the fine-tuning phase, the model's weights are optimized using a composite loss function. YOLOv8 calculates the total loss as a weighted sum of three distinct components:

1. **Box Loss** (`box_loss`): Measures the spatial alignment between the predicted bounding box and the ground truth. YOLOv8 utilizes Complete Intersection over Union (CIoU) loss. CIoU accounts

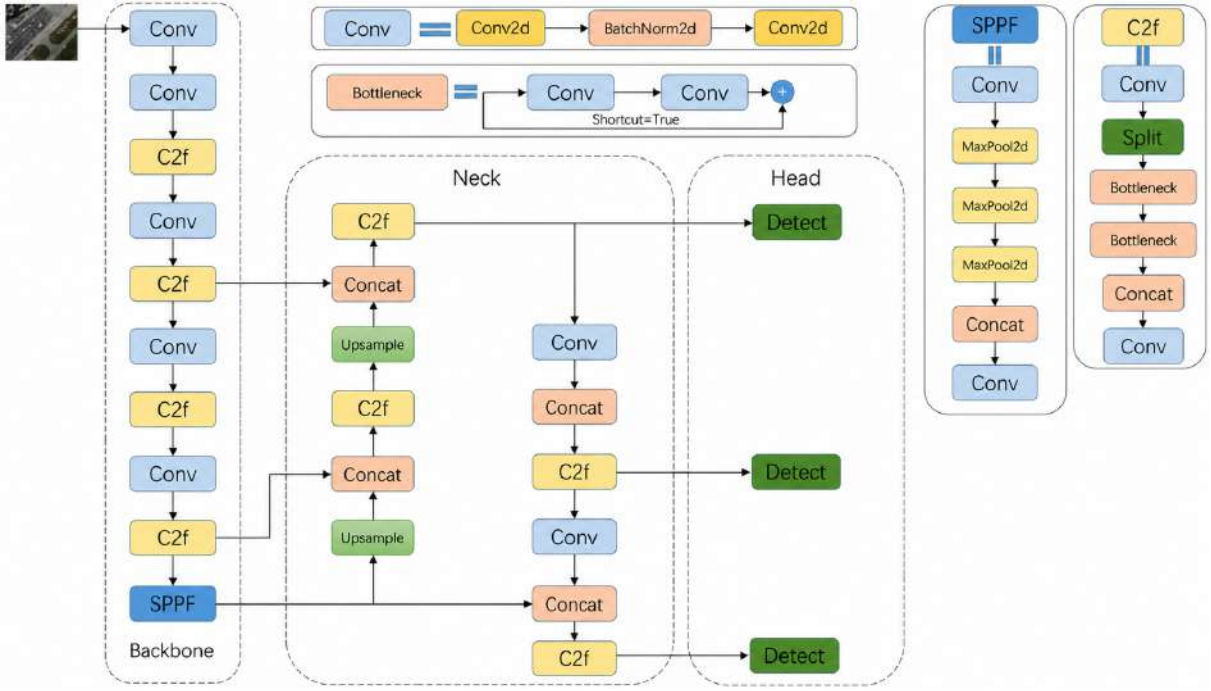


Figure 4.4: The original YOLOv8 architecture, showcasing the backbone with C2f modules, the Feature Pyramid Network (FPN)-Path Aggregation Network (PAN) neck for multi-scale feature fusion, and the decoupled head for parallel classification and regression. Reprinted from [50].

for the overlap area, the distance between box centers, and aspect ratio consistency, ensuring stable convergence for variable box shapes [52].

2. **Classification Loss** (`cls_loss`): Measures the accuracy of the class prediction. Because this thesis focuses on a single-class detection task (strobili), the Binary Cross-Entropy (BCE) loss is utilized to penalize incorrect class probabilities.
3. **Distribution Focal Loss** (`dfl_loss`): Addresses the localization ambiguity of bounding box edges. Instead of treating bounding box coordinates as deterministic absolute values, DFL treats them as a continuous probability distribution, enabling the model to achieve tighter localizations around small or heavily occluded objects.

4.3.3 Slicing Aided Hyper Inference (SAHI)

Standard CNNs typically accept fixed-size inputs (e.g., 640×640 pixels). Downscaling the native 5472×3648 pixel aerial images to this dimension degrades the 20-pixel strobili to an unidentifiable state. To maintain the native resolution during the evaluation phase, the SAHI framework is integrated into the pipeline. SAHI is model-agnostic and provides a scalable solution to the small object detection gap in large remote sensing images [38].

During inference, SAHI dynamically partitions the high-resolution query image into a grid of overlapping slices (configured to 640×640 pixels with a 20% overlap). An independent forward pass of the trained YOLOv8 model is executed on each slice. The resulting bounding box predictions are subsequently mapped back to their corresponding coordinates on the original macro-image.

Because the slicing mechanism forces overlapping regions, a single strobilus located at a tile seam may generate multiple detection boxes. To resolve this, a global NMS algorithm is applied across the reconstructed macro-image, filtering out redundant bounding boxes based on a defined IoU overlap threshold.

4.4 Implementation Environment

This section outlines the dataset composition, the software frameworks, and the hardware specifications used to execute and evaluate both system architectures.

4.4.1 Dataset Splitting and Tiling

The primary dataset consists of 60 high-resolution aerial images (5472×3648 pixels) paired with manually annotated binary ground-truth masks. To prevent data leakage and ensure unbiased evaluation, the dataset split was strictly enforced at the macro-image level, prior to any tiling operations. The 60 images were partitioned into 70% training (42 images), 15% validation (9 images), and 15% test (9 images) subsets.

During pre-processing, the Canopy ROI filter was applied. This allowed the system to discard background patches lacking foliage, a selective tiling strategy that significantly reduces computational overhead and minimizes training noise [39, 53]. This selective process yielded a final YOLO-formatted dataset comprising 1,337 training tiles, 280 validation tiles, and 290 test tiles, all normalized to a 640×640 pixel resolution.

4.4.2 Data Augmentation Configuration

To prevent overfitting on the limited dataset, a robust augmentation suite was configured during the YOLOv8 fine-tuning phase. The applied transformations simulate varying drone flight dynamics and forest illumination constraints:

- **Geometric Transformations:** Random rotations up to 180° , horizontal and vertical flips (50% probability), image scaling ($\pm 50\%$), translations (10%), and slight perspective shifts (0.0005) and shears (5.0°) [9].

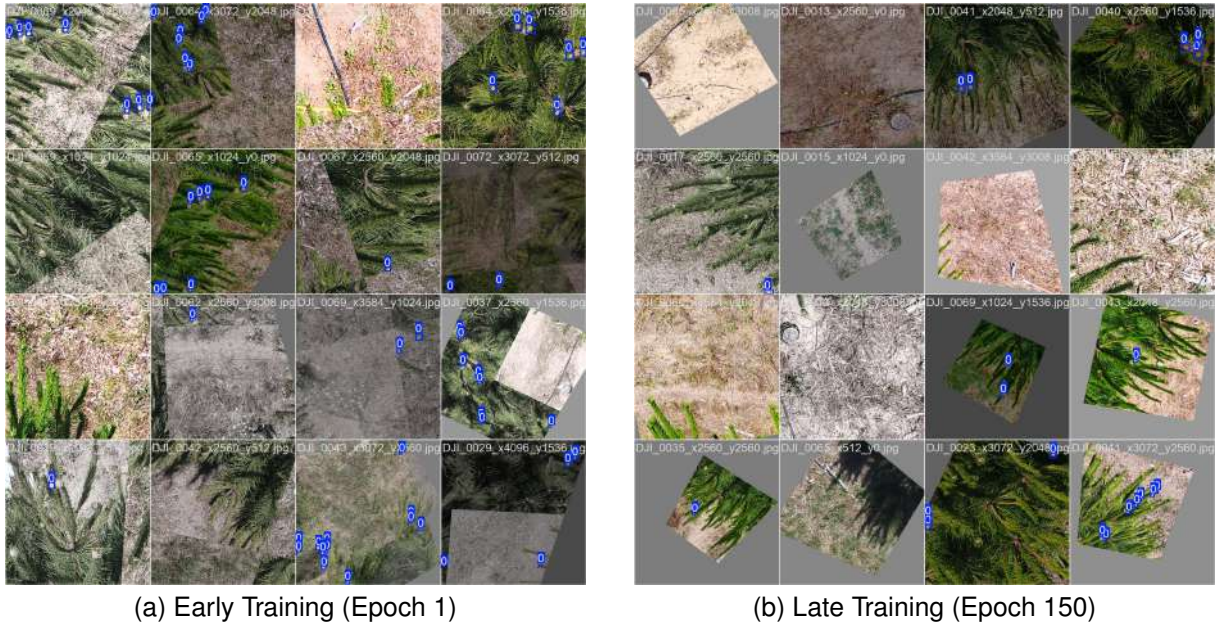


Figure 4.5: Visualization of the training batches and augmentation strategy. (a) shows the initial training phase with a batch size of 16, utilizing heavy Mosaic and Mixup augmentations to increase scene complexity. (b) depicts the final stabilization phase where Mosaic is disabled (`close_mosaic=10`), allowing the model to fine-tune on the natural spatial distribution of the pine needles and strobili.

- **Photometric Transformations:** Random adjustments to the HSV color space, including Hue ($\pm 3\%$), Saturation ($\pm 70\%$), and Value/Brightness ($\pm 50\%$) [42].
- **Mosaic and Mixup:** Mosaic augmentation (100% probability) was used to stitch four separate training tiles into a single grid, artificially increasing scene complexity [10, 54]. Mixup (15% probability) blended multiple images together to improve the detection of heavily occluded targets. Mosaic augmentation was disabled for the final 10 epochs (`close_mosaic=10`) to allow the model to stabilize on natural image distributions (see Figure 4.5).

4.4.3 Software and Hardware Specifications

The implementation was developed entirely in Python 3.14.2. The classical pipeline relies on the `OpenCV` and `NumPy` libraries for image manipulation. The deep learning pipeline uses the `Ultralytics` API for YOLOv8 model training and validation, alongside the `SAHI` library for sliced inference. Experiment tracking and metric logging were formatted using `JSON` and `CSV` standard libraries.

All deep learning training and inference processes were executed on a workstation equipped with an **NVIDIA GeForce GTX 1080 Ti** GPU (11 GB Video Random Access Memory (VRAM)). Due to the VRAM constraints of the hardware, the batch size for fine-tuning was restricted to 16 images per gradient update (visualized in Figure 4.5).

Table 4.1: Summary of implementation environment specifications.

Component	Specification
Hardware	NVIDIA GeForce GTX 1080 Ti (11 GB VRAM)
Programming Language	Python 3.14.2
Computer Vision Libraries	OpenCV, NumPy
Deep Learning Frameworks	Ultralytics (YOLOv8), SAHI
Base Image Resolution	5472×3648 pixels
Tile Resolution	640×640 pixels
Batch Size	16
Epochs	150

Under this hardware configuration, the YOLOv8 fine-tuning process was completed over 150 epochs. The YOLOv8-Nano variant converged in approximately 54 minutes, while the more parameter-dense YOLOv8-Small variant converged in 1 hour and 26 minutes. To support the reproducibility of this research, the complete source code, configuration files, and fine-tuned models are publicly available at <https://github.com/joaopinto6/pine-cone-detection>.

5

Evaluation and Results

Contents

5.1	Evaluation Protocol	31
5.2	Classical Pipeline Optimization	32
5.3	Deep Learning Fine-Tuning Results	34
5.4	Comparative Analysis	38

This chapter presents the quantitative evaluation of the two system architectures developed for pine strobili detection. Section 5.1 defines the evaluation protocol and dataset partitioning. Section 5.2 details the grid search optimization and performance of the classical heuristic pipeline. Section 5.3 reports the fine-tuning results and operational parameter optimization for the deep learning architectures. Finally, Section 5.4 provides a comparative analysis of the architectures, focusing on detection accuracy and computational inference latency.

5.1 Evaluation Protocol

To ensure unbiased evaluation and prevent data leakage, the dataset was partitioned at the macro-image level prior to any pre-processing or tiling. The complete dataset of 60 high-resolution aerial images was

divided into three mutually exclusive subsets:

- **Training Set (70%):** 42 images utilized exclusively for neural network weight optimization.
- **Validation Set (15%):** 9 images utilized for hyperparameter tuning and model selection during the training epochs.
- **Test Set (15%):** 9 images held out from all training and optimization phases. This set serves as the definitive benchmark for evaluating the final generalization capability of the pipelines. This 15% allocation aligns with findings by Kienbaum et al., who established that holding out 10 to 20% of an image dataset provides sufficient diversity in lighting and morphological characteristics for robust evaluation [55].

The performance of the detection pipelines was quantified using Precision, Recall, and the F1-score, as defined in Section 2.3. For the deep learning models, the mean Average Precision at an IoU threshold of 0.5 (mAP@50) was recorded during the training phase, following the established standard for object detection evaluation [22].

5.2 Classical Pipeline Optimization

The classical heuristic pipeline relies on static parameters. An extensive grid search was conducted to optimize each of the three primary processing stages, aiming to balance detection accuracy (Recall) with noise rejection (Precision/Coverage). Because the classical pipeline does not rely on learned weights, the evaluation was conducted across the entire dataset to ascertain the theoretical performance ceiling of the heuristics.

5.2.1 Region of Interest (ROI) Tuning

The objective of the ROI generation stage was to isolate the tree canopy from the background terrain while ensuring a target retention rate of 100% (no strobili erroneously cropped out). The grid search evaluated varying HSV boundaries, morphological closing kernel sizes (from 9 to 17) to balance noise removal with shape fidelity [56], and dilation iterations.

The optimal configuration yielded 100% target retention while reducing the image coverage to 27.4% of the original area. The optimal parameters were: Hue range [30, 80], Saturation [50, 255], Value [50, 255], closing kernel size of 9, and 2 dilation iterations.

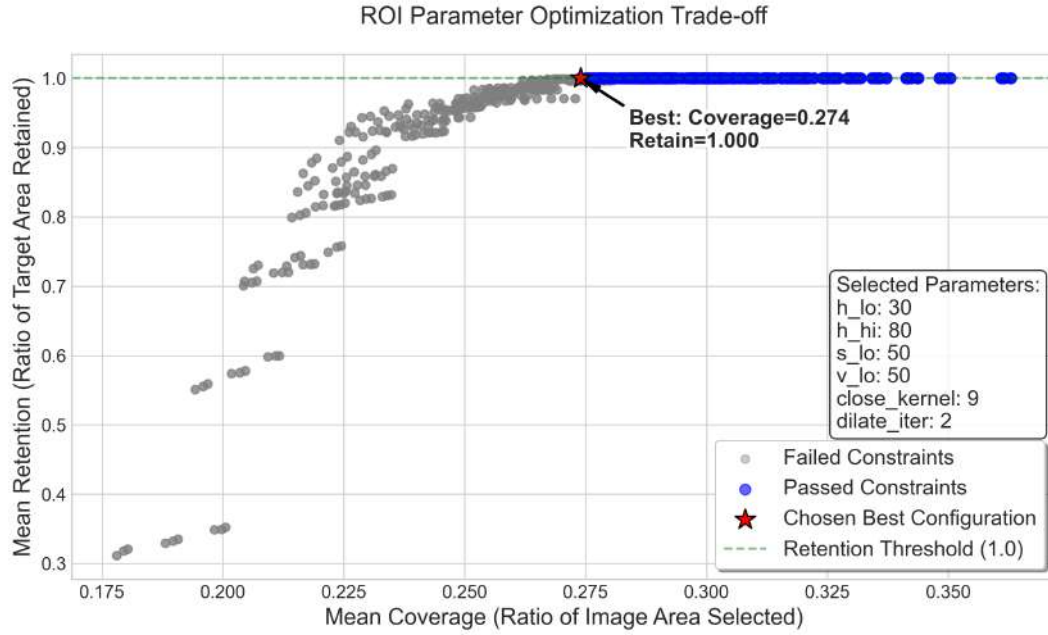


Figure 5.1: ROI optimization: Mean image coverage versus target retention. The optimal configuration point highlights the lowest possible coverage that maintains a retention threshold of 1.0.

5.2.2 Segmentation Tuning

The segmentation stage aimed to isolate strobili pixels within the defined canopy ROI. The grid search evaluated multiple color spaces (HSV and YCrCb) to identify the most robust representation against variable outdoor lighting [12, 32]. The search also evaluated lower and upper channel boundaries and morphological strategies (shapes: ellipse, rectangle; sizes: 3, 5, 7, 9; strategies: close-only, open-close).

The highest performance was achieved using the HSV color space with boundaries set to Lower: [20, 50, 80] and Upper: [35, 255, 255]. The optimal morphological refinement utilized an elliptical kernel of size 3 with a close-only strategy. This configuration yielded 100% instance recall with a highly restricted image coverage of 3.6%.

5.2.3 Candidate Proposal Optimization

The final classical tuning stage optimized the sliding-window generation and multi-template verification logic. The parameter space included sliding-window sizes (9 to 19), strides, pixel density thresholds (0.05 to 0.2), template correlation score thresholds (0.5 to 0.8), and NMS overlap thresholds. Evaluating NMS IoU thresholds is important for defining accurate positive detections and suppressing redundant bounding boxes [19].

The optimal configuration utilized a window size of 15, a stride of 2, a density threshold of 0.2, a template score threshold of 0.8, and an NMS overlap of 0.3. This configuration achieved the maximum

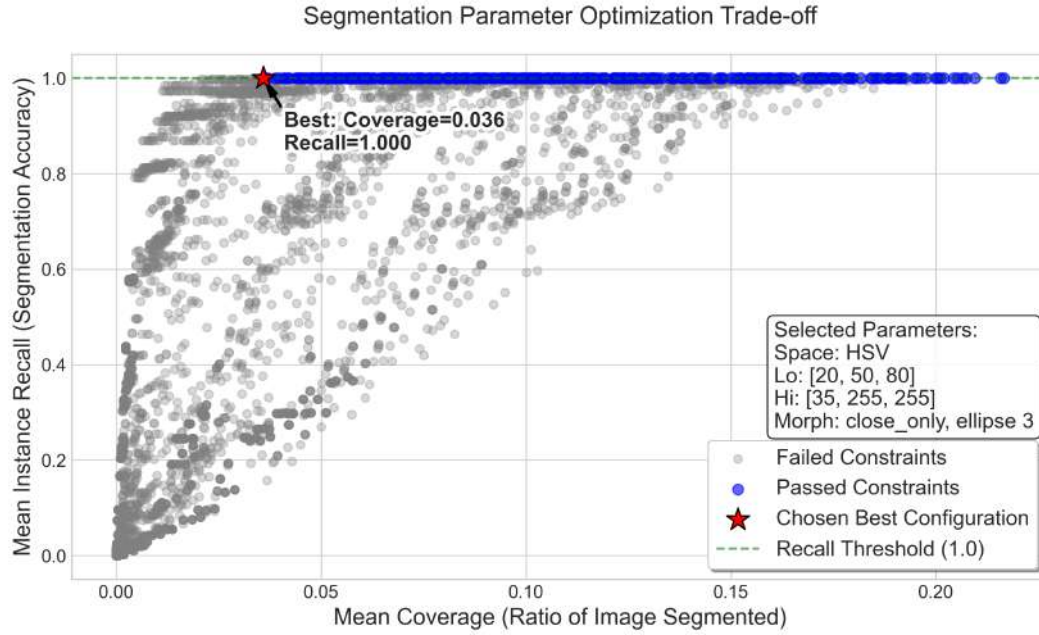


Figure 5.2: Segmentation optimization: Mean image coverage versus mean instance recall, demonstrating the separation capability of the optimized HSV color space.

overall performance for the classical pipeline, yielding a Precision of 0.717, a Recall of 0.327, and an F1-score of 0.449.

To visually contextualize the performance of the optimized classical pipeline, Figure 5.3 illustrates the final detection output on a sample image. The visualization highlights the categorization of bounding boxes evaluated against the ground-truth annotations, alongside the intermediate candidate proposals.

5.3 Deep Learning Fine-Tuning Results

Two deep learning architectures, YOLOv8-Nano (yolov8n) and YOLOv8-Small (yolov8s), were fine-tuned for 150 epochs using the tiled dataset. This section reports the training convergence and the subsequent optimization of operational inference parameters.

5.3.1 Training Convergence and Stability

During the fine-tuning phase, the mAP@50 metric was tracked to evaluate convergence.

YOLOv8-Small: The 11.2 million parameter model exhibited a sharp spike in validation loss during the initial epochs, indicative of early gradient instability common in larger models adapting to high learning rates. However, post-spike, the validation loss smoothed and converged closely with the training loss, indicating robust generalization. The model achieved its global best mAP@50 of 0.972 at Epoch

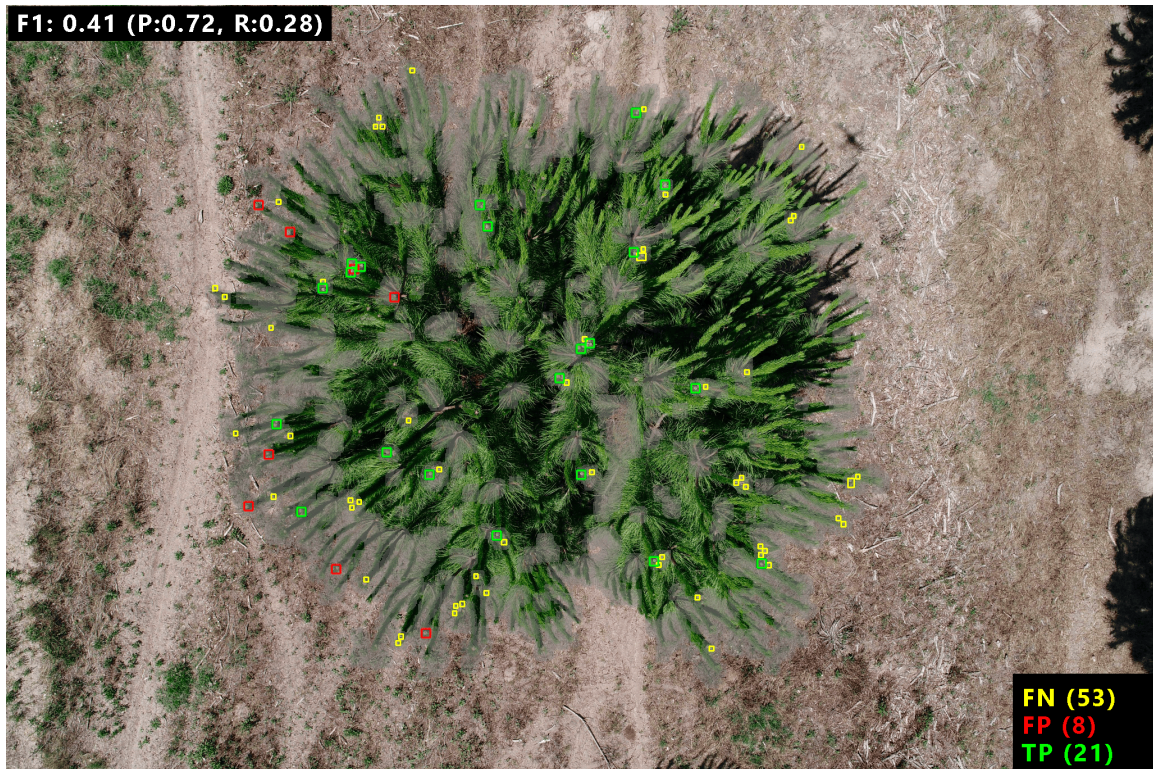


Figure 5.3: Visual representation of the classical pipeline's detection performance on a sample image. The bounding boxes indicate True Positives (green), False Positives (red), and False Negatives (yellow). The light gray overlay represents the initial sliding-window proposals generated prior to template verification and NMS filtering.

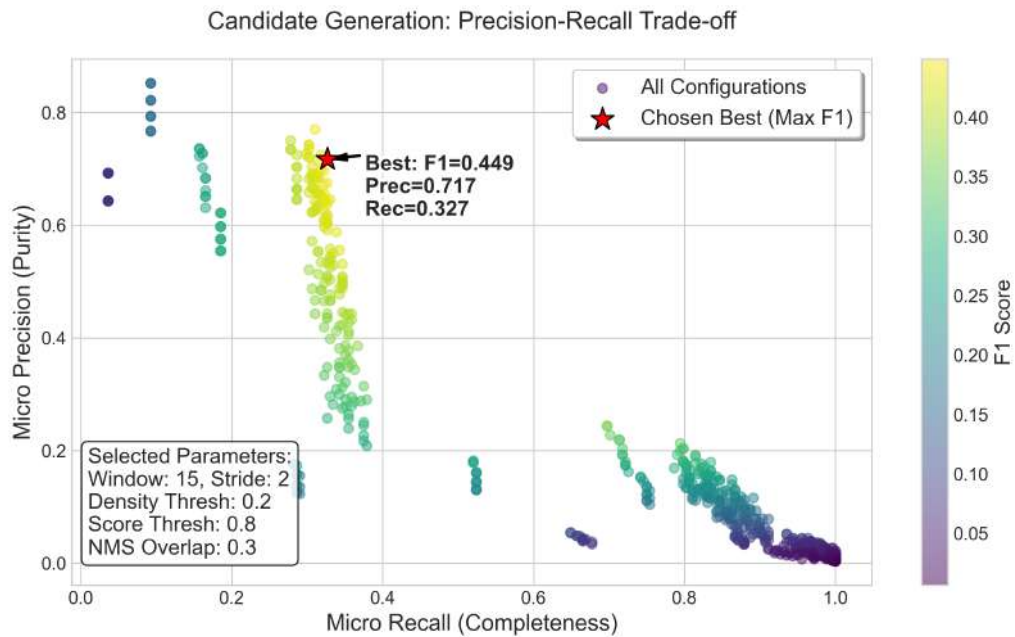


Figure 5.4: Candidate proposal optimization: Precision-Recall scatter plot with an F1-score color gradient, illustrating the trade-off between false positives and missed targets in the classical pipeline.

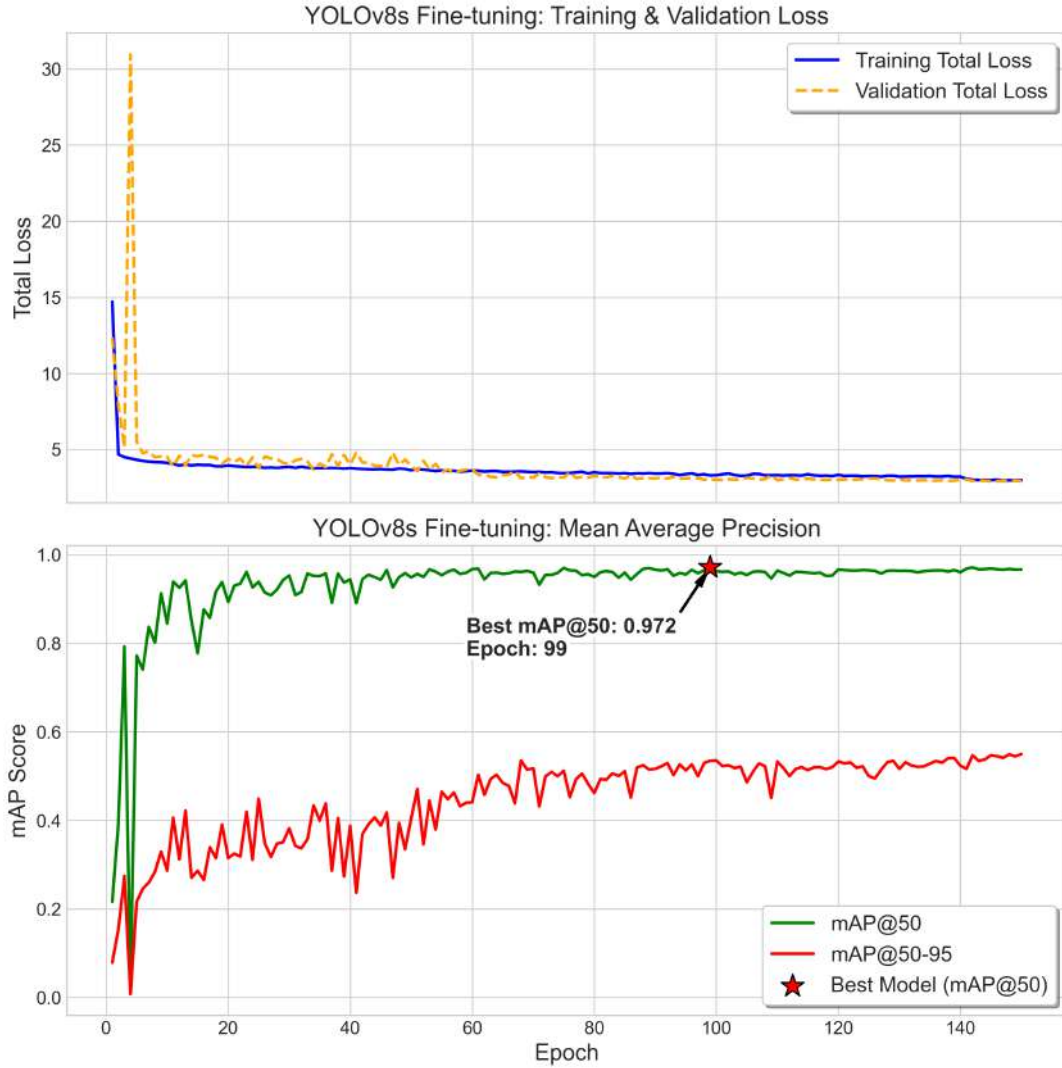


Figure 5.5: Training metrics for YOLOv8-Small over 150 epochs, displaying the loss convergence and the stabilization of the mAP@50 metric.

99, after which the metric plateaued, justifying the termination at 150 epochs.

YOLOv8-Nano: The 3.2 million parameter model demonstrated higher initial stability, with a consistent drop in loss from the first epoch. The validation loss remained lower than the training loss throughout the run, suggesting strong generalization against the augmented training data. The model achieved a peak mAP@50 of 0.975 at Epoch 102.

5.3.2 Operational Parameter Optimization

Following model training, a secondary grid search was performed to optimize the inference parameters for the SAHI framework [7]. The search space evaluated the Confidence Threshold [0.1, 0.6], IoU Match

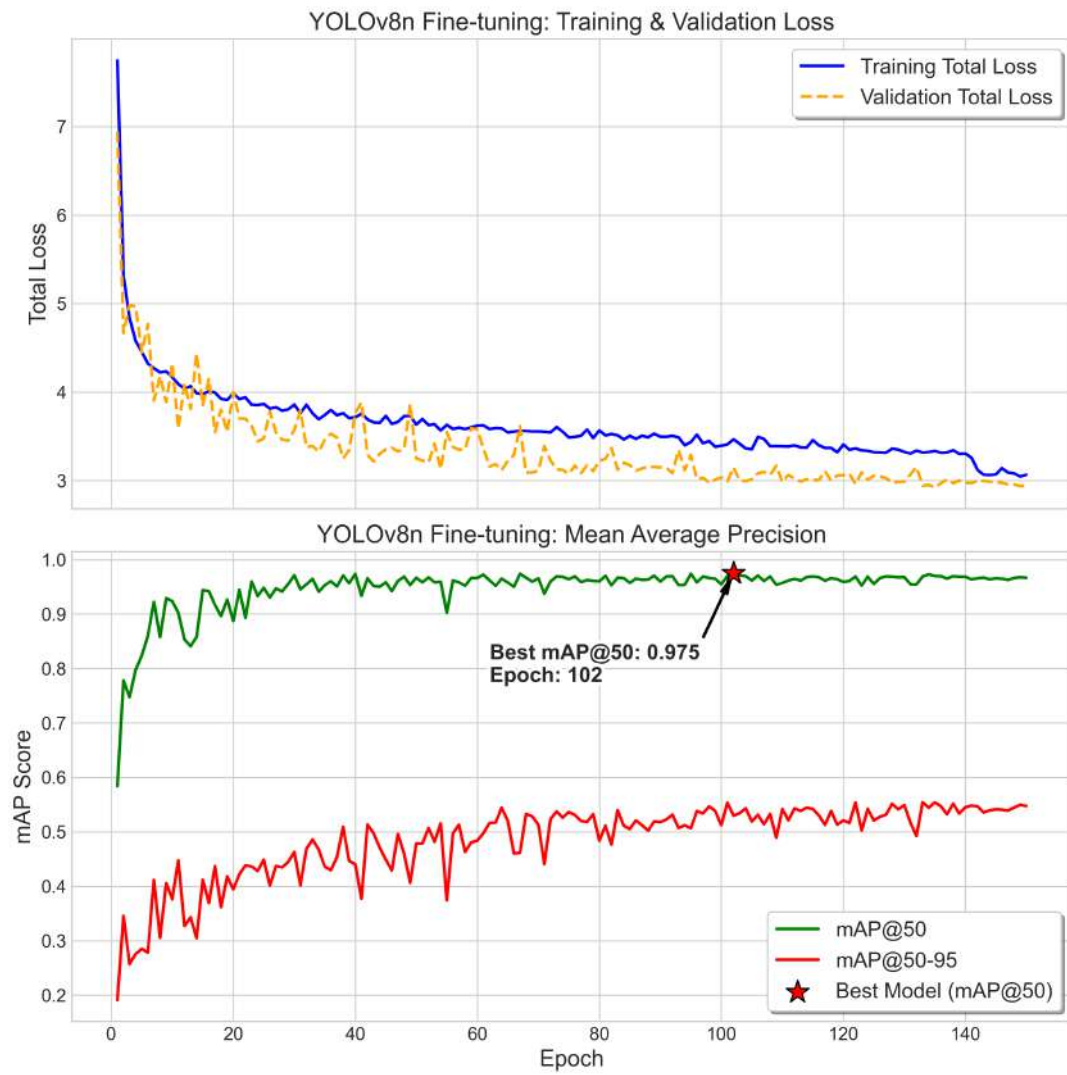


Figure 5.6: Training metrics for YOLOv8-Nano over 150 epochs, showing early convergence and the progression of the mAP@50 metric.

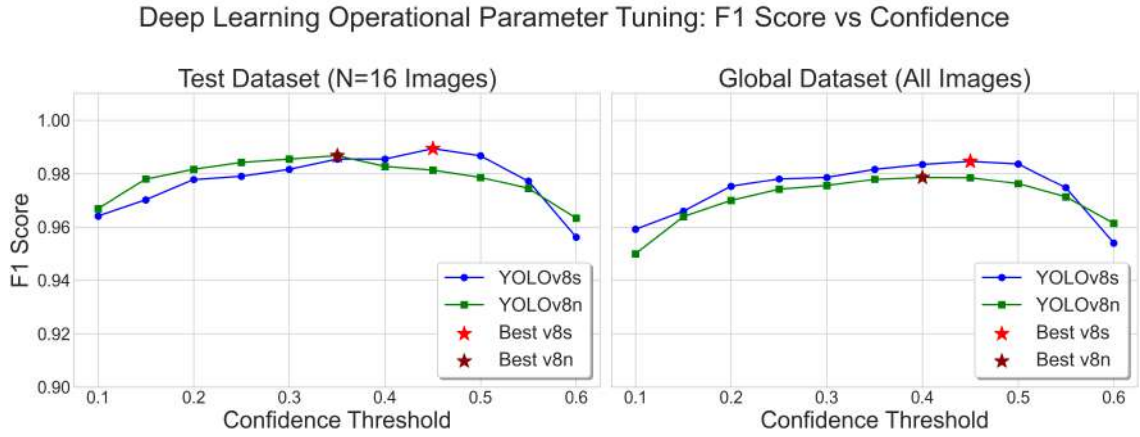


Figure 5.7: F1-score performance curves across varying confidence thresholds for YOLOv8-Small and YOLOv8-Nano, indicating the mathematically optimal operating points on the Test set (left) and Global dataset (right).

Threshold $[0.1, 0.4]$, and NMS IoU Threshold $[0.1, 0.4]$. The objective was to identify the configuration that maximized the F1-score.

Both architectures exhibited high stability, consistently achieving F1-scores above 0.95 across the majority of the testing range. The mathematically optimal points for both architectures on the independent Test set and the Global dataset are summarized in Table 5.1.

Table 5.1: Optimal inference parameters for deep learning architectures.

Dataset	Model	Conf. Threshold	IoU Match	NMS IoU	Max F1
Global	YOLOv8-Small	0.45	0.1	0.2	0.9846
Global	YOLOv8-Nano	0.40	0.1	0.3	0.9786
Test	YOLOv8-Small	0.45	0.1	0.1	0.9894
Test	YOLOv8-Nano	0.35	0.1	0.2	0.9868

The operational effectiveness of the optimized deep learning pipeline is visually represented in Figure 5.8. The figure displays the final aggregated bounding boxes mapped onto a full-scale test image following SAHI inference, demonstrating the system's capability to accurately detect spatially dense strobi.

5.4 Comparative Analysis

The final evaluation compares the theoretical and operational performance of the tested architectures.

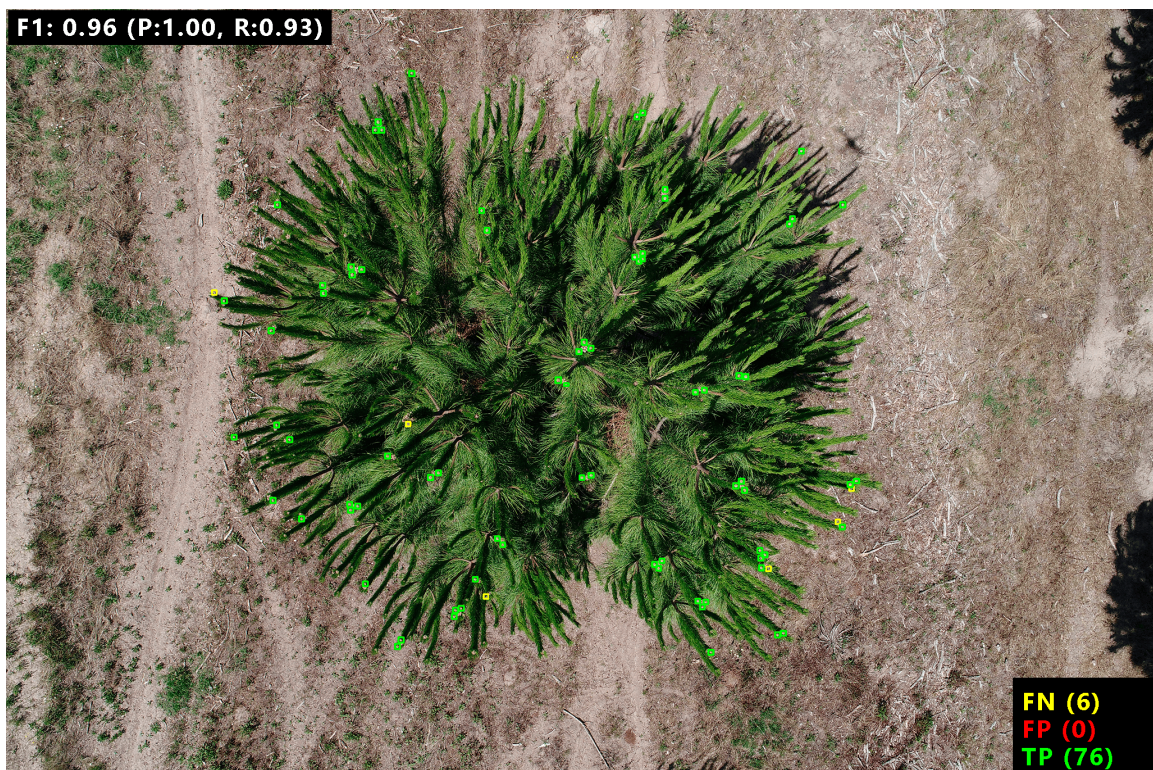


Figure 5.8: Visual representation of the deep learning pipeline's detection performance on a sample test image following SAHI inference. Bounding boxes are color-coded to indicate True Positives (green), False Positives (red), and False Negatives (yellow).

5.4.1 Detection Accuracy Comparison

The classical heuristic pipeline achieved a maximum F1-score of 0.449 on the global dataset. In contrast, the integration of deep learning architectures yielded a significant increase in detection accuracy, with both YOLOv8 variants surpassing a 0.97 F1-score.

Between the two neural networks, the YOLOv8-Small architecture consistently outperformed the Nano variant. On the hold-out Test set, YOLOv8-Small achieved an F1-score of 0.9894, compared to 0.9868 for the Nano model. This performance gap was maintained on the Global dataset (0.9846 vs. 0.9786), indicating that the increased parameter count of the Small architecture provides a more robust semantic representation of the strobili.

5.4.2 Computational Inference Latency

To evaluate the operational throughput, an inference benchmark was conducted using all 60 high-resolution aerial images integrated with the SAHI framework. To simulate a low-resource forestry deployment scenario typical of edge computing environments [39], the benchmark was executed on a consumer-grade CPU-based machine.

The YOLOv8-Nano model demonstrated significantly higher computational efficiency, completing inference in an average of 10.81 seconds per image. The YOLOv8-Small model required an average of 26.42 seconds per image, representing a 144% increase in computational latency.

Table 5.2: Inference latency benchmark (CPU-based execution).

Model	Avg. Inference Time (s)	Std. Deviation (s)
YOLOv8-Nano	10.81	1.77
YOLOv8-Small	26.42	3.74

While the YOLOv8-Small architecture offers the highest detection accuracy, the YOLOv8-Nano model provides an alternative operational profile, processing the dataset approximately $2.4\times$ faster while maintaining an F1-score exceeding 0.97.

6

Conclusions

Contents

6.1 Summary of Findings	41
6.2 Limitations and Future Work	42

This chapter summarizes the contributions and findings of this thesis in relation to the research objectives. Section 6.1 provides a synthesis of the implemented architectures and their respective performances. Section 6.2 discusses the current limitations of the system and outlines directions for future research and operational deployment.

6.1 Summary of Findings

The primary objective of this thesis was to develop an automated computer vision pipeline capable of detecting *Pinus pinea* strobili in high-resolution UAV imagery with an F1-score exceeding 0.90. By identifying these early-stage reproductive structures at the canopy apices, the system provides forestry managers with a three-year predictive window for harvest planning and resource allocation.

To address this challenge, a two-phase methodology was executed. Initially, a classical heuristic-based pipeline was developed using HSV color-space thresholding, canopy ROI filtering, and multi-

template matching. While computationally efficient, this deterministic approach reached a performance ceiling, achieving a maximum F1-score of 0.449. The classical pipeline struggled with the dynamic illumination, spectral overlap, and dense occlusion characteristic of natural forest environments, confirming that handcrafted features are insufficient for highly camouflaged, small-scale targets.

To overcome these feature extraction constraints, the methodology transitioned to a deep learning paradigm utilizing the YOLOv8 architecture. To address the "small object detection gap" inherent in aerial imagery, the pipeline incorporated a dataset tiling strategy and the SAHI framework. This allowed the neural network to process imagery at its native resolution, preventing the downscaling of the 20-pixel strobili.

The deep learning integration exceeded the initial research objective. The YOLOv8-Small architecture achieved an F1-score of 0.9894 on the independent test set, demonstrating strong localization and classification accuracy. A comparative analysis with the YOLOv8-Nano architecture (which achieved an F1-score of 0.9868) revealed a clear operational trade-off. While the Small variant provides the highest precision for rigorous counting tasks, the Nano variant processes images approximately 2.4 times faster. Consequently, the choice of model determines whether a deployment prioritizes maximum analytical accuracy or rapid, large-scale inference throughput.

Ultimately, this thesis delivers a robust, scalable, and automated workflow for precision forestry. The combination of domain-specific pre-processing (Canopy ROI filtering) and sliced deep learning inference addresses the multi-scale occlusion challenges of UAV-based strobili detection.

6.2 Limitations and Future Work

Despite the high detection accuracy achieved, the current system has operational limitations. The pipeline relies on high-resolution 5K imagery, which requires substantial storage capacity and low-altitude UAV flights. Furthermore, the reliance on the SAHI framework introduces a high computational overhead during inference, as a single full-resolution image must be sliced and processed through the neural network multiple times.

To address these limitations and expand the capabilities of the proposed system, several avenues for future work are identified:

- **Lightweight Edge Deployment:** Future iterations should explore model quantization techniques (such as TensorRT) to deploy the efficient YOLOv8-Nano architecture directly onto edge devices, such as the NVIDIA Jetson platform. Processing data onboard the UAV during flight would eliminate the need for post-flight data transfers and enable real-time yield estimation in the field.
- **Temporal and Multi-Temporal Analysis:** The current pipeline analyzes static snapshots of the canopy. Future research could incorporate multi-temporal UAV surveys, capturing images of the

same orchards across different seasons and years. Tracking the morphological growth of specific strobili over time would help validate the three-year maturity cycle and improve long-term predictive yield models.

- **Instance Segmentation vs. Object Detection:** While bounding boxes provide an accurate count of strobili, they lack precise dimensional data. Transitioning the architecture from object detection to instance segmentation (e.g., utilizing YOLOv8-seg) would output exact polygonal masks for each strobilus. This pixel-level morphological data could be used to automatically estimate the exact size and maturity stage of the developing cones.
- **Dataset Expansion and Generalization:** To ensure the commercial viability of the pipeline, the training dataset should be expanded to include diverse geographical locations, varied weather conditions, and potentially other pine species. Training on a more heterogeneous dataset would further solidify the model's resilience to edge-case environmental factors.

Bibliography

- [1] A. C. Correia, A. Farinha, J. E. Silva, A. Nunes, N. Conceição, M. da Encarnação Marcelo, A. Sarmento, M. Tomé, J. Soares, and L. Fontes, "Fertirrigation in grafted pinus pinea l. trees: denser crowns but no effect on cone production or masting cycles," *Forest Ecology and Management*, vol. 569, p. 122164, 2024.
- [2] B. Darwin, P. Dharmaraj, S. Prince, D. E. Popescu, and D. J. Hemanth, "Recognition of bloom/yield in crop images using deep learning models for smart agriculture: A review," *Agronomy*, vol. 11, no. 4, p. 646, 2021.
- [3] A. C. Correia, P. Amorim, P. Barcik, I. Carrasquinho, D. Lloberas Lafuente, M. d. E. Marcelo, C. Nogueira, P. Silveira, C. S. Trindade, and T. Valdivieso, "Capítulo - pinhão," in *Produtos Diretos com Fileira estabelecida: O PINHÃO*. Oeiras, Portugal: INIAV - Instituto Nacional de Investigação Agrária e Veterinária, I.P., 2025, trabalho realizado no âmbito do projeto CORKNUT (PCIF/MOS/0012/2019) e Agenda Transform.
- [4] G. Tang, J. Ni, Y. Zhao, Y. Gu, and W. Cao, "A survey of object detection for uavs based on deep learning," *Remote Sensing*, vol. 16, no. 1, p. 149, 2023.
- [5] Z. Wang and R. Boesch, "Color-and texture-based image segmentation for improved forest delineation," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 45, no. 10, pp. 3055–3062, 2007.
- [6] H. Khorsand, S. Arezoomandan, and D. K. Han, "Enhanced long-range uav detection: Leveraging slicing aided hyper inference with yolov8," in *2025 IEEE International Conference on Consumer Electronics (ICCE)*. IEEE, 2025, pp. 1–6.
- [7] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing aided hyper inference and fine-tuning for small object detection," in *2022 IEEE international conference on image processing (ICIP)*. IEEE, 2022, pp. 966–970.
- [8] U. Nisa, "Image augmentation approaches for small and tiny object detection in aerial images: A review," *Multimedia Tools and Applications*, vol. 84, no. 19, pp. 21 521–21 568, 2025.

- [9] Y. Diez, S. Kentsch, M. Fukuda, M. L. L. Caceres, K. Moritake, and M. Cabezas, "Deep learning in forestry using uav-acquired rgb data: A practical review," *Remote Sensing*, vol. 13, no. 14, p. 2837, 2021.
- [10] Z. Luo, Y. Zhang, K. Wang, and L. Sun, "Detection of pine cones in natural environment using improved yolov4 deep learning algorithm," *Computational Intelligence and Neuroscience*, vol. 2021, no. 1, p. 5601414, 2021.
- [11] K. B. Shaik, P. Ganesan, V. Kalist, B. Sathish, and J. M. M. Jenitha, "Comparative study of skin color detection and segmentation in hsv and ycbcr color space," *Procedia Computer Science*, vol. 57, pp. 41–48, 2015.
- [12] E. Prasetyo, R. D. Adityo, N. Suciati, and C. Fatichah, "Mango leaf image segmentation on hsv and ycbcr color spaces using otsu thresholding," in *2017 3rd International Conference on Science and Technology-Computer (ICST)*. IEEE, 2017, pp. 99–103.
- [13] I. Abbas, J. Liu, M. Faheem, R. S. Noor, S. A. Shaikh, K. A. Solangi, and S. M. Raza, "Different sensor based intelligent spraying systems in agriculture," *Sensors and Actuators A: Physical*, vol. 316, p. 112265, 2020.
- [14] A. Wang, W. Zhang, and X. Wei, "A review on weed detection using ground-based machine vision and image processing techniques," *Computers and electronics in agriculture*, vol. 158, pp. 226–240, 2019.
- [15] P. Sadeghi-Tehran, N. Virlet, K. Sabermanesh, and M. J. Hawkesford, "Multi-feature machine learning model for automatic segmentation of green fractional vegetation cover for high-throughput field phenotyping," *Plant methods*, vol. 13, no. 1, p. 103, 2017.
- [16] K. Yu, N. Kirchgessner, C. Grieder, A. Walter, and A. Hund, "An image analysis pipeline for automated classification of imaging light conditions and for quantification of wheat canopy cover time series in field phenotyping," *Plant methods*, vol. 13, no. 1, p. 15, 2017.
- [17] J. Tian, D. Panangian, W. Fan, B. Siegmann, and X. Yuan, "Deep learning based individual tree crown delineation from panchromatic aerial imagery," *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, pp. 885–892, 2025.
- [18] M. Possolo and P. Bajcsy, "Exact tile-based segmentation inference for images larger than gpu memory," *Journal of Research of the National Institute of Standards and Technology*, vol. 126, p. 126009, 2021.

- [19] H. Rezatofighi, T. T. D. Nguyen, B.-N. Vo, B.-T. Vo, S. Savarese, and I. Reid, "How trustworthy are performance evaluations for basic vision tasks," *arXiv preprint arXiv:2008.03533*, vol. 2, no. 3, p. 8, 2020.
- [20] Z. Alyafeai and L. Ghouti, "A fully-automated deep learning pipeline for cervical cancer classification," *Expert Systems with Applications*, vol. 141, p. 112951, 2020.
- [21] D. Müller and F. Kramer, "Miscnn: a framework for medical image segmentation with convolutional neural networks and deep learning," *BMC medical imaging*, vol. 21, pp. 1–11, 2021.
- [22] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft coco: Common objects in context," in *European conference on computer vision*. Springer, 2014, pp. 740–755.
- [23] H. S. Ndlovu, J. Odindi, M. Sibanda, and O. Mutanga, "A systematic review on the application of uav-based thermal remote sensing for assessing and monitoring crop water status in crop farming systems," *International Journal of Remote Sensing*, vol. 45, no. 15, pp. 4923–4960, 2024.
- [24] D. Radočaj, A. Šiljeg, R. Marinović, and M. Jurišić, "State of major vegetation indices in precision agriculture studies indexed in web of science: A review," *Agriculture*, vol. 13, no. 3, p. 707, 2023.
- [25] R. Vidican, A. Mălinaş, O. Ranta, C. Moldovan, O. Marian, A. Gheţe, C. R. Ghişo, F. Popovici, and G. M. Căţunescu, "Using remote sensing vegetation indices for the discrimination and monitoring of agricultural crops: a critical review," *Agronomy*, vol. 13, no. 12, p. 3040, 2023.
- [26] Y. Wang, Z. Yang, G. Kootstra, and H. A. Khan, "The impact of variable illumination on vegetation indices and evaluation of illumination correction methods on chlorophyll content estimation using uav imagery," *Plant Methods*, vol. 19, no. 1, p. 51, 2023.
- [27] P. Seralathan and S. Edward, "Revolutionizing agriculture: Deep learning-based crop monitoring using uav imagery-a review," in *2024 International Conference on Expert Clouds and Applications (ICOECA)*. IEEE, 2024, pp. 777–784.
- [28] H. Zhu, C. Lin, G. Liu, D. Wang, S. Qin, A. Li, J.-L. Xu, and Y. He, "Intelligent agriculture: Deep learning in uav-based remote sensing imagery for crop diseases and pests detection," *Frontiers in Plant Science*, vol. 15, p. 1435016, 2024.
- [29] Q. Feng, X. Xu, and Z. Wang, "Deep learning-based small object detection: A survey," *Mathematical Biosciences and Engineering*, vol. 20, no. 4, pp. 6551–6590, 2023.
- [30] J. Butler and H. Leung, "A novel keypoint supplemented r-cnn for uav object detection," *IEEE Sensors Journal*, vol. 23, no. 24, pp. 30 883–30 892, 2023.

- [31] M. F. Humayun, A. Salmasi, J. Zhang, T. Westerlund, and J. Heikkonen, "A robust integrated approach for near real-time seamless orthomosaic generation using off-the-shelf uavs for ultra-high resolution mapping applications," *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. 10, pp. 65–72, 2025.
- [32] M. A. Khan and M. A. AlGhamdi, "An intelligent and fast system for detection of grape diseases in rgb, grayscale, ycbcr, hsv and l* a* b* color spaces," *Multimedia Tools and Applications*, vol. 83, no. 17, pp. 50 381–50 399, 2024.
- [33] J. Martinez-Garcia, I. Stelzner, J. Stelzner, D. Gwerder, and P. Schuetz, "Automated 3d tree-ring detection and measurement from x-ray computed tomography," *Dendrochronologia*, vol. 69, p. 125877, 2021.
- [34] J. M. Miller, "Considerations using iterative closest point in presence of occlusions in automated aerial refueling," Master's Thesis, Air Force Institute of Technology, Wright-Patterson Air Force Base, Ohio, 2022, aFIT-ENG-MS-22-M-047.
- [35] J. Wang, J. Gao, and B. Zhang, "A small object detection model in aerial images based on cpdd-yolov8," *Scientific Reports*, vol. 15, no. 1, p. 770, 2025.
- [36] S. Zeng, W. Yang, Y. Jiao, L. Geng, and X. Chen, "Sca-yolo: A new small object detection model for uav images," *The visual computer*, vol. 40, no. 3, pp. 1787–1803, 2024.
- [37] K. Niu and Y. Yan, "A small-object-detection model based on improved yolov8 for uav aerial images," in *2023 2nd International Conference on Artificial Intelligence and Intelligent Information Processing (AIIIP)*. IEEE, 2023, pp. 57–60.
- [38] F. M. A. Mazen and Y. Shaker, "Small object detection in complex images: Evaluation of faster r-cnn and slicing aided hyper inference," *Int. J. Adv. Comput. Sci. Appl*, vol. 16, no. 3, 2025.
- [39] G. Plastiras, S. Siddiqui, C. Kyrkou, and T. Theocharides, "Efficient embedded deep neural-network-based object detection via joint quantization and tiling," in *2020 2nd IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. IEEE, 2020, pp. 6–10.
- [40] Y. Zhu, G. Chen, L. Zhang, J. Zhou, J. Kuang, and J. Zhu, "Lightweight insulator defect detection in high-resolution uav imagery via system-level co-design," *Remote Sensing*, vol. 18, no. 6, p. 953, 2026.
- [41] P. Kaur, B. S. Khehra, and E. B. S. Mavi, "Data augmentation for object detection: A review," in *2021 IEEE International Midwest Symposium on Circuits and Systems (MWSCAS)*. IEEE, 2021, pp. 537–543.

- [42] M. N. A. Yehia, S. M. M. Elfeky, and A. Hamdi, "Enhanced drone-based plant disease detection with visual data augmentation and optimized training," in *2024 International Mobile, Intelligent, and Ubiquitous Computing Conference (MIUCC)*. IEEE, 2024, pp. 478–483.
- [43] K. M. Gishyan, "Improving uav object detection through image augmentation," *Mathematical Problems of Computer Science*, vol. 54, pp. 53–68, 2020.
- [44] W. Hua and Q. Chen, "A survey of small object detection based on deep learning in aerial images," *Artificial Intelligence Review*, vol. 58, no. 6, p. 162, 2025.
- [45] H. Pei, T. Owari, S. Tsuyuki, and Y. Zhong, "Application of a novel multiscale global graph convolutional neural network to improve the accuracy of forest type classification using aerial photographs," *Remote Sensing*, vol. 15, no. 4, p. 1001, 2023.
- [46] E. Yildirim, M. Nazar, U. G. Sefercik, and T. Kavzoglu, "Stone pine (*pinus pinea* l.) detection from high-resolution uav imagery using deep learning model," in *IGARSS 2022-2022 IEEE International Geoscience and Remote Sensing Symposium*. IEEE, 2022, pp. 441–444.
- [47] A. S. Saif, P. Paul, K. M. Zubair, S. A. Shubho, and D. Nandi, "Traffic sign detection based on color segmentation of obscure image candidates: a comprehensive study," *International Journal of Modern Education and Computer Science*, vol. 10, no. 6, pp. 35–46, 2019.
- [48] J. Funck, Y. Zhong, D. Butler, C. Brunner, and J. Forrer, "Image segmentation algorithms applied to wood defect detection," *Computers and electronics in agriculture*, vol. 41, no. 1-3, pp. 157–179, 2003.
- [49] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics yolov8," 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [50] H. Zhang, J. Yang, W. Zhang, and J. Ren, "lcm-yolo: Uav remote sensing small target detection algorithm based on feature fusion enhancement module and ghost convolution," *Physica Scripta*, vol. 99, no. 12, p. 125036, 2024.
- [51] B. Karacaoglu and M. E. Sahin, "Optimized yolo architectures for efficient kiwi detection in precision agriculture on embedded systems," *Scientific Reports*, 2025.
- [52] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-iou loss: Faster and better learning for bounding box regression," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 07, 2020, pp. 12 993–13 000.
- [53] M. M. Afsar, A. D. Bakhshi, M. S. Iqbal, E. Hussain, and J. Iqbal, "High-precision mango orchard mapping using a deep learning pipeline leveraging object detection and segmentation," *Remote Sensing*, vol. 16, no. 17, p. 3207, 2024.

- [54] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "Yolov4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.
- [55] L. Kienbaum, M. Correa Abondano, R. Blas, and K. Schmid, "Deepcob: Precise and high-throughput analysis of maize cob geometry using deep learning with an application in genebank phenomics," *Plant Methods*, vol. 17, no. 1, p. 91, 2021.
- [56] A. K. Kanithi and S. Meher, "Study of spatial and transform domain filters for efficient noise reduction," *Department of Electronics and Communication Engineering, National Institute of Technology, Rourkela, India*, 2011.



Source Code Excerpts

This appendix provides relevant excerpts from the Python codebase developed for this thesis. The complete source code, including data indexing, pre-processing, and visualization modules, is publicly available in the GitHub repository referenced in Section 4.4.3.

A.1 YOLOv8 Fine-Tuning Configuration

Listing A.1 presents the core function utilized to fine-tune the YOLOv8 architectures. The configuration highlights the extensive geometric and photometric augmentations applied to simulate drone flight variability and forest illumination constraints.

Listing A.1: YOLOv8 fine-tuning configuration with advanced data augmentation.

```
1 def train_yolo(  
2     dataset_yaml: Path,  
3     model_size: str,  
4     epochs: int,  
5     imgsz: int,  
6     batch: int,
```

```

7     device: str,
8     project_dir: Path,
9     run_name: str,
10    workers: int,
11    seed: int,
12    use_wandb: bool,
13 ) -> Dict:
14     from ultralytics import YOLO
15
16     if use_wandb:
17         os.environ.setdefault("WANDB_PROJECT", "pine-cone-yolo")
18         os.environ.setdefault("WANDB_NAME", run_name)
19
20     model = YOLO(_resolve_model_checkpoint(model_size))
21
22     results = model.train(
23         data=str(dataset_yaml),
24         epochs=epochs,
25         imgsz=imgsz,
26         batch=batch,
27         device=device,
28         workers=workers,
29         project=str(project_dir),
30         name=run_name,
31         seed=seed,
32         pretrained=True,
33         cache=False,
34         val=True,
35         save=True,
36         plots=True,
37         verbose=True,
38         # Geometric and Photometric Augmentations
39         degrees=180.0,
40         translate=0.10,
41         scale=0.50,
42         shear=5.0,
43         perspective=0.0005,
44         fliplr=0.5,

```

```

45         flipud=0.5,
46         hsv_h=0.03,
47         hsv_s=0.70,
48         hsv_v=0.50,
49         mosaic=1.0,
50         mixup=0.15,
51         copy_paste=0.0,
52         close_mosaic=10,
53     )
54
55     save_dir = getattr(results, "save_dir", None)
56     return {"run_dir": str(save_dir)}

```

A.2 SAHI Inference Implementation

Listing A.2 details the integration of the SAHI framework for high-resolution sliced inference. This function dynamically partitions the native 5472×3648 pixel aerial images into overlapping 640×640 patches, executing the YOLOv8 model independently on each slice before aggregating the predictions via NMS.

Listing A.2: SAHI sliced inference integration for high-resolution macro-images.

```

1  def run_sahi_inference_on_test(
2      model_path: Path,
3      manifest_csv: Path,
4      test_ids_path: Path,
5      dataset_scope: str,
6      slice_size: int,
7      overlap_ratio: float,
8      device: str,
9      nms_iou_threshold: float,
10 ) -> Dict[str, Dict[str, List]]:
11     from sahi import AutoDetectionModel
12     from sahi.predict import get_sliced_prediction
13
14     eval_entries = _load_entries_by_scope(
15         manifest_csv=manifest_csv,
16         dataset_scope=dataset_scope,

```

```

17         test_ids_path=test_ids_path,
18     )
19
20     detection_model = AutoDetectionModel.from_pretrained(
21         model_type="ultralytics",
22         model_path=str(model_path),
23         confidence_threshold=0.001,
24         device=device,
25     )
26
27     per_image = {}
28     for entry in eval_entries:
29         gt_bboxes_abs, _, _ = mask_to_bboxes(entry.mask_path,
30             min_contour_area=1.0)
31
32         gt_xyxy = [_to_xyxy_from_gt(x, y, w, h) for (x, y, w, h) in
33             gt_bboxes_abs]
34
35         prediction_result = get_sliced_prediction(
36             image=str(entry.image_path),
37             detection_model=detection_model,
38             slice_height=slice_size,
39             slice_width=slice_size,
40             overlap_height_ratio=overlap_ratio,
41             overlap_width_ratio=overlap_ratio,
42             postprocess_type="NMS",
43             postprocess_match_metric="IOU",
44             postprocess_match_threshold=nms_iou_threshold,
45             verbose=0,
46         )
47         pred_boxes = _extract_sahi_prediction_boxes(prediction_result)
48
49         per_image[entry.image_id] = {
50             "gt_xyxy": gt_xyxy,
51             "pred_boxes": pred_boxes,
52             "image_path": str(entry.image_path),
53         }
54
55     return per_image

```

A.3 Evaluation and Metric Calculation

Listing A.3 outlines the core evaluation logic used to compare the model's predictions against the manual ground-truth annotations. The function employs greedy IoU matching to categorize bounding boxes into True Positives, False Positives, and False Negatives, enabling the calculation of Precision, Recall, and the F1-score across varying confidence thresholds.

Listing A.3: Evaluation logic for computing Precision, Recall, and F1-score via IoU matching.

```
1 def evaluate_predictions(  
2     per_image_predictions: Dict[str, Dict[str, List]],  
3     conf_thresholds: Sequence[float],  
4     iou_match_thresholds: Sequence[float],  
5     nms_iou_threshold: float,  
6 ) -> List[MetricsRow]:  
7     rows: List[MetricsRow] = []  
8  
9     for iou_match_threshold in iou_match_thresholds:  
10         for conf_th in conf_thresholds:  
11             total_tp = 0  
12             total_fp = 0  
13             total_fn = 0  
14             all_matched_iou: List[float] = []  
15  
16             for image_data in per_image_predictions.values():  
17                 gt_xyxy = image_data["gt_xyxy"]  
18                 # Filter predictions by confidence threshold  
19                 pred_boxes = [box.xyxy for box in image_data["pred_boxes"] if  
20                     box.score >= conf_th]  
21  
22                 # Perform greedy IoU matching  
23                 tp, fp, fn, matched_iou = _greedy_match(  
24                     pred_boxes=pred_boxes,  
25                     gt_boxes=gt_xyxy,  
26                     iou_match_threshold=iou_match_threshold,  
27                 )  
28                 total_tp += tp  
29                 total_fp += fp
```

```

29         total_fn += fn
30         all_matched_iious.extend(matched_iious)
31
32         # Calculate aggregate metrics
33         precision = total_tp / (total_tp + total_fp) if (total_tp +
34             total_fp) > 0 else 0.0
35         recall = total_tp / (total_tp + total_fn) if (total_tp + total_fn
36             ) > 0 else 0.0
37         f1 = (2.0 * precision * recall) / (precision + recall) if (
38             precision + recall) > 0 else 0.0
39         mean_iou = float(np.mean(all_matched_iious)) if all_matched_iious
40             else 0.0
41
42         rows.append(
43             MetricsRow(
44                 conf_threshold=float(conf_th),
45                 iou_match_threshold=float(iou_match_threshold),
46                 nms_iou_threshold=float(nms_iou_threshold),
47                 true_positives=total_tp,
48                 false_positives=total_fp,
49                 false_negatives=total_fn,
50                 precision=precision,
51                 recall=recall,
52                 f1=f1,
53                 mean_iou=mean_iou,
54             )
55         )
56
57     return rows

```